

Trabajo Académico PR2

Diseño de una solución de integración en el ámbito de una fábrica

Grado en Informática Industrial y Robótica
2º curso

Valentina Monteserín
Claudia Castillo Suárez
Ainhoa García Herrera
Rihab Baitar

Índice general

1. Introducción	1
1.1. Introducción	1
1.2. Presentación del proyecto y equipo	1
1.3. Descripción del funcionamiento	2
1.3.1. Proceso por automatizar	2
1.3.2. Medidas a considerar	3
1.3.3. Proceso antes de ser automatizado	3
2. Propuesta de automatización	5
2.1. Estructura del trabajo	5
2.1.1. Simulación del proceso mediante la página web	5
2.1.2. Persistencia en base de datos y publicación MQTT (primer puente Python)	5
2.1.3. Activación de RoboDK mediante el segundo puente Python	6
2.1.4. Segunda planta: sensores externos, ESP32 y seguridad simulada	6
2.2. Objetivos, impacto y limitaciones del proyecto	7
3. Diseño de la solución	9
3.1. Layout de la planta	9
3.2. Pagina web; Base de datos; puente mqtt	11
3.2.1. Diseño e implementación	11
3.2.2. Por qué se utiliza JavaScript	12
3.2.3. Partes relevantes del código JavaScript	12
3.2.4. Relación web – MQTT – base de datos	14
3.2.5. Datos de cliente guardados en base de datos	14
3.2.6. Datos de pedido guardados en base de datos	15
3.2.7. Resumen del flujo completo desde la web	15
3.2.8. Explicación detallada del <i>mqtt_{bridge}</i>	15
3.2.9. Parte 1: utilidades de texto y tipo de pedido: <i>MQTT_{bridge}</i>	17
3.2.10. Parte 2: cálculo del número de combinación ; <i>mqtt_{bridge}</i>	17
3.2.11. Parte 3: preparación del esquema de base de datos	17
3.2.12. Parte 4: procesamiento del carrito y stock	18
3.2.13. Parte 5: gestión del cliente (<i>Mi cuenta</i>)	18
3.2.14. Parte 6: guardado completo de un pedido	19
3.2.15. Parte 7: conexión MQTT	20
3.2.16. Parte 8: función <i>main()</i> – arranque del servicio	21
3.2.17. Tabla resumen de topics MQTT	22
3.2.18. Tabla resumen: qué se guarda en PostgreSQL	23

3.3.	Puente MQTT_station_relay	23
3.3.1.	Rol en el sistema	23
3.3.2.	Configuración (.env)	23
3.3.3.	Doble conexión MQTT	24
3.3.4.	Extracción del número de combinación	24
3.3.5.	Reenvío hacia RoboDK	24
3.3.6.	Ejecución	25
3.4.	Sistemas de seguridad con ESP32	25
3.4.1.	Elementos físicos usados	25
3.4.2.	Código Arduino	26
3.4.3.	MQTT_listener en RoboDK	27
3.5.	Programacion en RoboDK	28
3.5.1.	Organización de las secuencias	28
3.5.2.	Estructuración de los procesos	29
3.5.3.	Desarrollo de las secuencias	30
3.5.4.	Sistema de señalización	30
3.5.5.	Relacion MQTT-python	31
4.	Cronograma y análisis financiero	34
4.1.	CRONOGRAMA y gestión del proyecto	34
4.1.1.	Pruebas y validación	35
4.2.	Costes y beneficios	35
5.	Normativa, regulación y seguridad	39
5.0.1.	Normativas y estándares aplicables	39
5.0.2.	Consideraciones de seguridad en planta	40
5.0.3.	Impacto en los puestos de trabajo	40
6.	Conclusiones	41
7.	Referencias	43
7.1.	Descripción de la implementación	43
7.2.	División en tareas y funciones	43
7.3.	Uso de la IA	44
7.4.	Referencias bibliográficas	44
8.	Anexos	47
8.1.	Anexo I. Manual de instalación y funcionamiento	47
8.1.1.	1. Requisitos previos	47
8.1.2.	2. Configuración del proyecto	47
8.1.3.	3. Instalación de dependencias	48
8.1.4.	4. Configuración de RoboDK	48
8.1.5.	5. Ejecución del puente MQTT	48
8.1.6.	6. Funcionamiento del sistema	48
8.1.7.	7. Verificación y resolución de problemas	49
8.2.	Anexo II. Relacion con PRA y GDI	49
8.3.	Anexo III. Puesta en marcha del sistema de sensores (ESP32 + RoboDK) .	50
8.3.1.	Componentes necesarios	50
8.3.2.	Flujo de comunicación	50

8.3.3.	Paso 1: Configurar el ESP32 (secrets.h)	51
8.3.4.	Paso 2: Librerías Arduino necesarias	51
8.3.5.	Paso 3: Subir el sketch al ESP32	51
8.3.6.	Paso 4: Configurar el PC (.env)	52
8.3.7.	Paso 5: Arrancar el puente industrial	52
8.3.8.	Paso 6: Iniciar RoboDK	53
8.3.9.	Paso 7: Orden de encendido recomendado	53
8.3.10.	Comportamiento esperado al probar sensores	53
8.3.11.	Solución de problemas	54
8.3.12.	Resumen	54
8.4.	Anexo IV	54

Capítulo 1

Introducción

1.1. Introducción

El presente proyecto se centra en el desarrollo de un sistema automatizado para el proceso de empaquetado y paletizado de mercancías. Para ello, se integran diversos dispositivos industriales, robots colaborativos y sistemas de comunicación, permitiendo la sincronización y coordinación de todas las etapas involucradas en el proceso productivo.

El objetivo principal del sistema es mejorar la eficiencia de la producción, tanto en la elaboración de pedidos estándar como de pedidos personalizados. Para alcanzar este propósito, se implementa una arquitectura basada en la comunicación e intercambio de información entre los distintos elementos que componen el sistema, mediante el uso de dispositivos embebidos ESP32 y software específico. Asimismo, se persigue la mejora de las condiciones laborales de los trabajadores de la planta, favoreciendo tanto su bienestar como el incremento de la productividad derivado de un entorno de trabajo más adecuado. Esto se debe a que las tareas repetitivas asociadas a este tipo de procesos suelen tener un impacto negativo sobre las condiciones físicas y psicológicas de los empleados.

Como consecuencia de la implementación de esta solución, se prevé obtener resultados positivos en términos de eficiencia operativa, reflejados en una reducción significativa de los tiempos de producción respecto a la situación previa. No obstante, también se contemplan diversos retos durante la puesta en marcha del proyecto, entre los que destacan el rediseño de la planta industrial, la inversión inicial requerida, los desafíos asociados al desarrollo del software y la correcta simulación y validación del sistema propuesto.

1.2. Presentación del proyecto y equipo

Smart Chocolate Factory es una empresa dedicada a la fabricación y comercialización de bombones, ofreciendo tanto pedidos personalizados como producción al por mayor para distintos tipos de clientes y empresas. La presente propuesta se enmarca dentro del área productiva de la organización, concretamente en el proceso de empaquetado y paletizado de las cajas de bombones. Según los análisis realizados por el departamento financiero, esta etapa constituye actualmente un cuello de botella dentro de la cadena de producción, generando un impacto negativo sobre el rendimiento anual de la empresa. Por este motivo, el Departamento de Automatización y Control plantea el desarrollo de un proyecto de automatización cuyo objetivo es reducir dichas limitaciones y optimizar el flujo operativo de la organización en su conjunto.

La coordinación general del proyecto fue asumida por Valentina Monteserín, quien desempeñó el papel de líder del equipo. Entre sus principales responsabilidades se encontraron la planificación y organización de las tareas, la supervisión del progreso de las diferentes áreas de trabajo, la coordinación entre los miembros del equipo y el seguimiento del cumplimiento de los objetivos establecidos durante el desarrollo de la solución propuesta.

En relación con la composición del equipo, cabe destacar que, aunque cada integrante contaba con un rol principal asignado, todas las participantes colaboraron de forma activa en las distintas fases del proyecto, contribuyendo de manera conjunta a su desarrollo e implementación. No obstante, los roles principales desempeñados fueron los siguientes:

- Ainhoa García Herrera: Ingeniera de Automatización y Control.
- Valentina Monteserín: Ingeniera de Robótica y Simulación.
- Claudia Castillo: Ingeniera de Puesta en Marcha.
- Rihab Baitar: Programadora de Sistemas OT.

Una vez presentado el equipo de trabajo, resulta importante justificar la necesidad de esta propuesta. El proceso manual de empaquetado y paletizado presenta diversas limitaciones, entre las que destacan los retrasos en los tiempos de producción, la aparición de errores humanos y el desperdicio de producto, factores que afectan negativamente a la continuidad y eficiencia de la cadena productiva. Con el fin de solucionar esta problemática, el proyecto plantea la automatización de dichas operaciones, demostrando que esta alternativa constituye una solución más eficiente. Su implementación no solo permitiría eliminar gran parte de las ineficiencias actuales, sino también incrementar el rendimiento global del sistema, mejorar la productividad y garantizar una mayor fiabilidad en el proceso de producción.

1.3. Descripción del funcionamiento

1.3.1. Proceso por automatizar

El proceso productivo se desarrolla en una fábrica dedicada a la comercialización de tres variedades de bombones: avellana, chocolate blanco y caramelo. Las cajas correspondientes a cada sabor acceden a la celda de trabajo mediante tres cintas transportadoras independientes. Durante su recorrido, cada caja es sometida a un proceso de control de calidad basado en el peso, verificando que su contenido se encuentre dentro del rango establecido de $200\text{ g} \pm 2,5$

Una vez superada esta etapa de validación, las cajas acceden al área de trabajo principal, donde operan dos robots colaborativos ABB CRB 15000. El primero de ellos se encarga de clasificar las cajas según su sabor e introducirlas en embalajes destinados a pedidos industriales o ventas al por mayor. Por su parte, el segundo robot está dedicado a la preparación de pedidos personalizados, configurando las cajas de acuerdo con las combinaciones de sabores solicitadas por los clientes.

Con el fin de evitar posibles colisiones dentro del espacio compartido de trabajo, cuando ambos robots requieran manipular cajas del mismo sabor, se establece una estrategia de alternancia en la recogida de producto. De este modo, cada robot accede al área de trabajo de forma secuencial hasta que uno de los dos finaliza su tarea correspondiente.

Cuando una caja de pedidos industriales alcanza su capacidad máxima, se marca automáticamente con un identificador que indica el sabor contenido en su interior. Posteriormente, la caja continúa su recorrido a través de la cinta transportadora destinada a pedidos industriales hasta llegar al área de trabajo del robot paletizador ABB IRB 4400/L30. Este robot identifica el tipo de producto mediante la lectura del marcador y deposita la caja en el palé correspondiente a dicho sabor.

En el caso de los pedidos personalizados, una vez que el robot colaborativo completa una caja, esta es transportada mediante una línea independiente. Al final de dicha línea, un sensor capacitivo detecta su llegada y envía una señal al robot paletizador. Como respuesta, el robot finaliza la operación de paletizado que esté ejecutando en ese momento, atiende prioritariamente la caja personalizada y, una vez completada esta tarea, reanuda el proceso de paletizado de pedidos industriales. Esta estrategia resulta viable debido al menor volumen de producción asociado a los pedidos personalizados, por lo que las interrupciones generadas no afectan significativamente al rendimiento global del sistema.

Finalmente, cuando un palé alcanza su capacidad máxima, una baliza luminosa se activa para indicar la necesidad de retirarlo y sustituirlo por uno vacío. En caso de que no se realice esta operación antes de la llegada de una nueva caja correspondiente al mismo tipo de producto, el sistema detiene automáticamente la producción hasta que el palé completo sea retirado y reemplazado, garantizando así la seguridad y continuidad del proceso.

1.3.2. Medidas a considerar

Cada caja de bombones tiene un peso de 200 g, pesando 15 g cada bombón y 35 g el envoltorio, y unas dimensiones de 20 cm × 10 cm × 4 cm. Conociendo esos parámetros y sabiendo que las medidas de las cajas de empaquetado son de 40 cm × 40 cm × 48 cm, caben un total de 96 cajas de bombones, 12 pisos de 8 cajas. Por tanto, en cada pallet europeo caben 24 cajas de empaquetado, lo que es equivalente a 9024 cajas de bombones.

Por otro lado, el peso final de las cajas se eleva a 19,2 kg sin contar con la caja de embalaje de triple pared, la cual hace que el peso total sea de 21,1 kg.

1.3.3. Proceso antes de ser automatizado

Antes de la implementación de los sistemas automatizados descritos en el proyecto, el proceso de paletizado de bombones se realizaba de manera completamente manual. En este procedimiento, el personal encargado debía recibir las cajas provenientes de las cintas transportadoras y comprobar de forma individual que cada una cumpliera con el peso estándar de $200\text{ g} \pm 2,5\%$, separando aquellas que presentaban defectos y depositándolas en áreas de rechazo.

Una vez comprobadas, las cajas se colocaban manualmente dentro de las cajas de embalado, 96 en total para completar una caja. Una vez ya se ha completado, se cierran de forma manual y etiquetan de ese mismo modo. Teniendo en cuenta el peso de las cajas, el personal habría de transportar cada caja de 21,2 kg hacia el pallet europeo por tal de completar un ciclo. Una vez ya colocada una caja, faltan otras 23 para poder retirar el pallet.

El tiempo estimado que conlleva cerrar una caja de embalaje es de 5 minutos sin contar distracciones. Añadiendo el transporte hacia el pallet y su correcta posición, 10 a 30 segundos según la posición, el tiempo total para completar un pallet es de 124–136

Capítulo 2

Propuesta de automatización

2.1. Estructura del trabajo

Para poder demostrar el funcionamiento integrado de la fábrica de bombones, el proyecto se ha organizado en varias capas conectadas entre sí: una capa de interfaz (página web), una capa lógica (puentes Python y base de datos), una capa de comunicación (MQTT con distintos brokers según el componente) y una capa física/simulada (RoboDK y ESP32 con sensores externos). Gracias a esta arquitectura distribuida, un pedido realizado desde el navegador puede recorrer todo el flujo hasta activar la simulación robotizada en RoboDK, mientras que, en paralelo, otra planta virtual permite demostrar funciones de seguridad industrial.

2.1.1. Simulación del proceso mediante la página web

Como punto de entrada del sistema se ha desarrollado una aplicación web estática (HTML, CSS y JavaScript) que simula la tienda online de la fábrica. Desde esta interfaz, el usuario puede navegar por el catálogo de bombones, configurar su pedido (industrial o personalizado) y gestionar los datos de su cuenta.

Antes de confirmar un pedido, la web solicita al cliente que inicie sesión o registre su cuenta, recogiendo la información necesaria según el tipo de cliente (autónomo o empresa). Además, durante el proceso de compra el usuario debe seleccionar la empresa de reparto entre las opciones disponibles, de modo que el pedido quede asociado tanto al cliente como a la logística de envío.

Cuando el cliente confirma el pedido, la propia página web publica un mensaje JSON en el broker MQTT (conexión WSS), en el topic de nuevos pedidos. De este modo, la web no necesita un servidor backend propio: actúa como un nodo más del sistema de mensajería y delega el procesamiento al resto de componentes.

2.1.2. Persistencia en base de datos y publicación MQTT (primer puente Python)

El script `mqtt_bridge.py` funciona como el backend lógico del sistema. Este puente permanece suscrito al topic de pedidos entrantes y, cada vez que la web publica un nuevo pedido:

- Valida y procesa la información recibida (cliente, líneas del pedido, tipo de pedido, envío, etc.).

- Guarda el pedido en la base de datos PostgreSQL, actualizando tablas de clientes, pedidos, líneas, envíos y stock de sabores.
- Publica mensajes de confirmación hacia la web (por ejemplo, confirmación del pedido o alertas de stock insuficiente).
- Asigna un número de combinación asociado al pedido (especialmente en pedidos personalizados), necesario para seleccionar la secuencia correcta en RoboDK.

Toda la configuración de conexión (credenciales de la base de datos, broker MQTT, topics, etc.) se centraliza en el archivo `.env`, cargado mediante `python-dotenv`. Así, el mismo código puede adaptarse a distintos entornos cambiando variables de entorno, sin modificar el programa. Gracias al `.env`, el puente sabe a qué broker conectarse, con qué usuario y contraseña, y en qué topics publicar o suscribirse (pedidos, confirmaciones, número de combinación, status de estación, alertas de stock, etc.).

En resumen, `mqtt_bridge.py` convierte un mensaje MQTT de la web en datos persistentes en SQL y, a continuación, vuelve a emitir eventos MQTT para que el resto del sistema reaccione.

2.1.3. Activación de RoboDK mediante el segundo puente Python

Una vez el pedido está registrado y se ha calculado el número de combinación de sabores, entra en juego el script `mqtt_station_relay.py`.

Este segundo puente actúa como intermediario entre el broker cloud (EMQX), usado por la web y el backend, y el broker MQTT de la UPV, al que se conecta RoboDK. Escucha el topic de estado de la estación (donde `mqtt_bridge.py` publica el número de combinación asignado) y, al recibirlo, reenvía ese número al topic de comandos de la estación de demostración:

```
giirob/pr2/station/demo/commands
```

RoboDK, mediante su programa `MQTT_listener.py`, permanece suscrito a ese topic. Cuando recibe el número de combinación, selecciona y ejecuta automáticamente la secuencia correspondiente dentro del proyecto simulado (movimientos de cintas, pick & place, paletizado, etc.). De esta forma, un pedido hecho desde la web desencadena el proceso automatizado en la planta virtual de RoboDK.

Este diseño fue necesario porque el broker de la universidad no permitía la conexión directa desde la aplicación web; por eso se implementaron puentes Python que adaptan la arquitectura de comunicación entre capas.

2.1.4. Segunda planta: sensores externos, ESP32 y seguridad simulada

Paralelamente al flujo web → base de datos → RoboDK, se ha montado otra planta virtual orientada a demostrar el sistema de seguridad y la interacción con sensores externos, utilizando un ESP32-S3 programado para publicar comandos MQTT.

Mediante el script `mqtt_industrial_bridge.py`, los mensajes generados por el ESP32 se reenvían al broker MQTT de la UPV, donde los recibe el listener de RoboDK. Así se simulan condiciones industriales reales de parada, ralentización e interrupción del proceso.

En concreto, se han implementado las siguientes funciones:

- **Control de velocidad de los robots colaborativos según proximidad:** un sensor ultrasónico HC-SR04 mide la distancia a objetos o personas en la zona de trabajo. Si detecta proximidad por debajo de un umbral, publica el comando `SLOW`; al alejarse, envía `NORMAL`. RoboDK interpreta estos mensajes y reduce o restaura la velocidad de los robots colaborativos, simulando el comportamiento exigido por normativas de robots colaborativos (como ISO/TS 15066).
- **Parada de emergencia del robot de paletizado:** mediante un botón físico conectado al ESP32 (comando `STOP`), se simula una parada de emergencia que detiene el proceso en RoboDK, equivalente a una seta de emergencia en planta real.
- **Interrupción para pedidos personalizados:** otro botón (comando `PER`) permite cortar el proceso industrial en curso y conmutar hacia la lógica de pedidos personalizados, demostrando cómo el sistema puede ser interrumpido y reorientado según la demanda del cliente.

2.2. Objetivos, impacto y limitaciones del proyecto

En cuanto a los objetivos del proyecto, la meta principal consiste en diseñar e implementar un sistema de automatización del proceso de empaquetado y paletizado de *Smart Chocolate Factory*, con el fin de mejorar la eficiencia productiva y el rendimiento global de la empresa. De forma más específica, se han definido los siguientes *Key Performance Indicators* (KPIs):

- Incrementar la capacidad de empaquetado en al menos un 40 % respecto al proceso manual.
- Reducir el tiempo medio de preparación de un pedido personalizado en un 50 %.
- Minimizar la tasa de error en el empaquetado por debajo del 2 %.
- Disponer de un sistema operativo y una simulación funcional antes de la feria de proyectos.

Además del proceso de empaquetado y paletizado, la propuesta tiene un impacto positivo en otras áreas del sistema productivo y de la organización. En particular, la automatización contribuye a la mejora de los procesos posteriores al paletizado, tales como el etiquetado, el almacenamiento y la distribución, al proporcionar un flujo de trabajo más rápido, una mayor trazabilidad y una mejor coordinación entre etapas.

Asimismo, el área de logística se ve beneficiada, ya que la automatización facilita la planificación de ciclos productivos, la estimación de tiempos y la optimización de recursos. En relación con el personal operativo, se plantea una reubicación de los trabajadores afectados hacia tareas de mayor valor añadido, como la supervisión del sistema automatizado o el control de calidad, con el objetivo de mantener su incorporación en la estructura organizativa y potenciar su contribución a procesos críticos.

El proyecto también presenta distintos impactos adicionales. Desde el punto de vista tecnológico, fomenta la adopción de robótica colaborativa y sistemas embebidos en entornos industriales. En el ámbito logístico, permite analizar mejoras en los tiempos de producción y entrega, así como una mayor eficiencia global del sistema. Desde una perspectiva económica, la automatización conlleva una reducción de costes operativos en

producción, lo que permite la redistribución de recursos hacia puestos de mayor valor estratégico. Finalmente, desde el punto de vista ambiental, el uso de dispositivos electrónicos, robots y sistemas automatizados implica un incremento del consumo energético y el uso de materiales potencialmente contaminantes, lo que debe ser considerado en el análisis global del sistema.

No obstante, la propuesta también presenta diversas limitaciones y restricciones que han condicionado su diseño. En primer lugar, el elevado coste de inversión inicial ha supuesto una limitación significativa, lo que ha obligado a optimizar la selección de robots, cintas transportadoras y sensores con el fin de ajustar el presupuesto disponible.

En segundo lugar, se debe tener en cuenta la necesidad de formación específica del personal, ya que la transición hacia un sistema automatizado requiere un periodo de adaptación, así como costes asociados a formación, paradas de producción y validación del sistema.

Por otro lado, en el ámbito de la implementación, las limitaciones en el desarrollo de la página web han condicionado la capacidad de presentar una interfaz completamente optimizada. Del mismo modo, el uso de *RoboDK* como entorno de simulación presenta restricciones en la modelización de sensores y en el nivel de realismo del comportamiento físico del sistema.

Finalmente, el uso del microcontrolador ESP32-S3 también introduce limitaciones relacionadas con la capacidad de procesamiento y la complejidad de los programas ejecutables, lo que ha requerido simplificar determinadas funcionalidades para garantizar su correcto funcionamiento.

Capítulo 3

Diseño de la solución

3.1. Layout de la planta

Como se ha descrito en el Capítulo 1, el proceso automatizado abarca alimentación por cintas, empaquetado, cerrado y paletizado. En este apartado no se repite ese flujo funcional, sino que se resume **qué equipos físicos lo componen** y cómo se integran en la planta simulada.

La celda diseñada en *RoboDK* se organiza en cuatro zonas, de izquierda a derecha, siguiendo el sentido del producto:

1. **Zona de alimentación:** tres cintas transportadoras en paralelo (avellana, chocolate blanco y caramelo), con control de peso y rechazo de cajas fuera de tolerancia.
2. **Zona de empaquetado:** dos robots colaborativos ABB CRB 15000 con pinzas de vacío, dedicados respectivamente a pedidos al por mayor y personalizados.
3. **Zona de transición:** cinta central de mayor anchura, máquina de cerrado y marcado láser.
4. **Zona de paletizado:** robot industrial ABB IRB 4400/L30, cuatro estaciones de pallet (tres industriales y una de pedido a demanda) y baliza de señalización.

Cuadro 3.1: Equipos principales de la celda automatizada

Elemento	Modelo / tipo	Rol en la celda
Robots de empaquetado	2 × ABB CRB 15000	Pick & place de cajas de bombones
Robot de paletizado	ABB IRB 4400/L30	Apilado sobre pallets
Transporte	3 cintas de entrada + 1 cinta central	Alimentación y traslado entre zonas
Control de calidad	Báscula + pistón neumático	Verificación de peso y rechazo
Identificación	Lector de códigos	Enrutado al pallet correspondiente
Señalización	Baliza PATLITE	Aviso de pallet completo

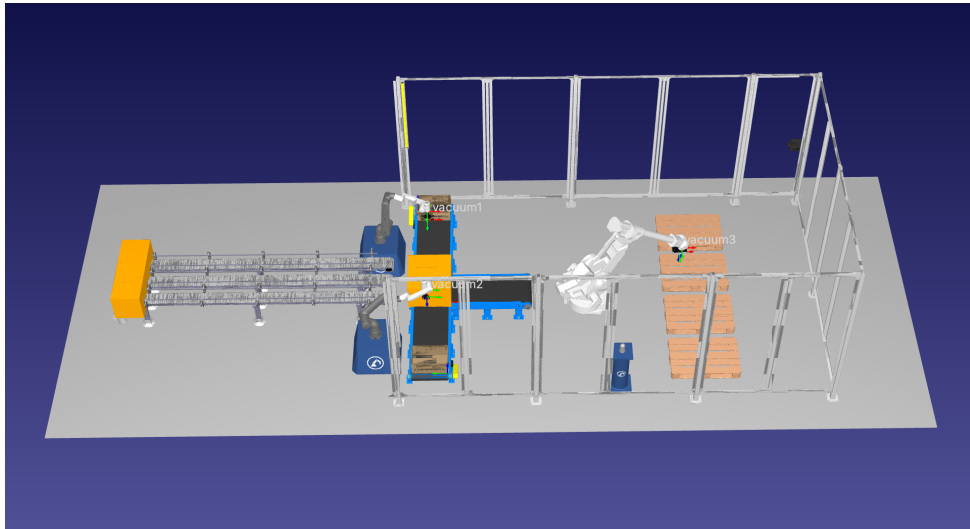


Figura 3.1: Planta PR2; implementación en *RoboDK*

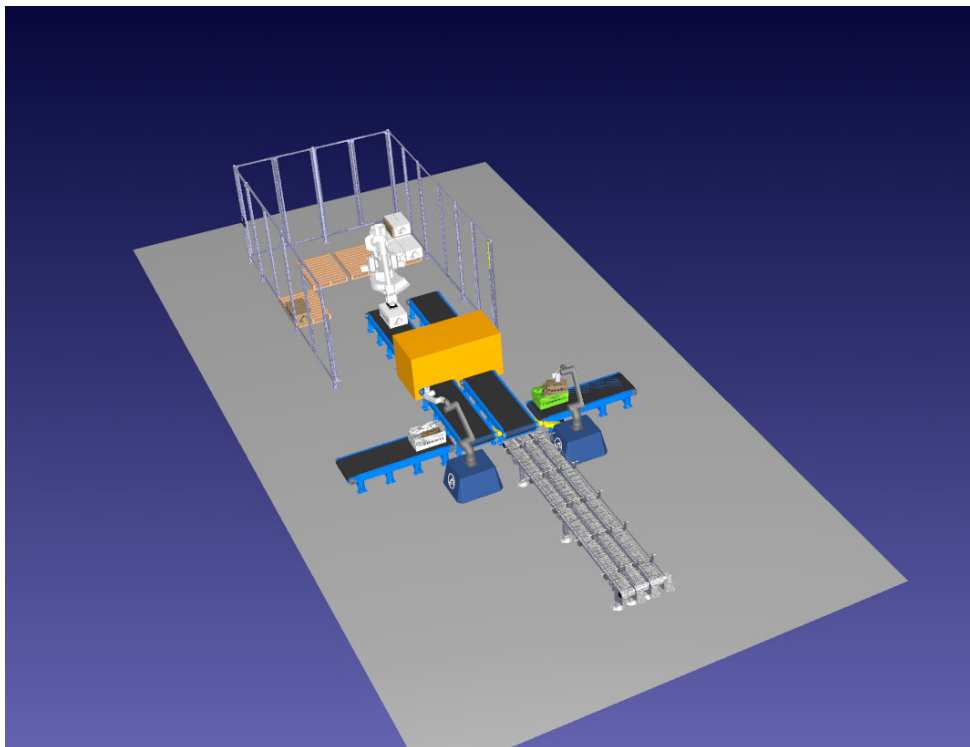


Figura 3.2: Planta PR2 con sistema de sensores

Actualmente se han definido dos diseños de planta: la implementación real, mostrada en la Figura 3.2, y la implementación final desarrollada en *RoboDK*, representada en la Figura 3.1.

Ambas versiones presentan diferencias debido a las limitaciones del entorno de simulación. En particular, durante el desarrollo del modelo en *RoboDK* se intentó incorporar una segunda cinta transportadora con el objetivo de reproducir fielmente la planta real; sin embargo, dicha implementación no fue viable debido a restricciones técnicas del software.

En consecuencia, la Figura 3.2 representa el diseño ideal del sistema en un entorno real, incluyendo la integración de los sistemas de seguridad. Por su parte, la Figura 3.1 corresponde a la versión adaptada en *RoboDK*, diseñada específicamente para garantizar la continuidad del flujo de trabajo y permitir la validación de todas las combinaciones posibles dentro del sistema simulado.

3.2. Pagina web; Base de datos; puente mqtt

La aplicación web de *Chocolat Suprême* constituye la capa de interfaz del sistema. Permite simular el comportamiento de una tienda online desde la que el cliente configura pedidos, gestiona su cuenta y dispara, mediante MQTT, el resto del flujo automatizado (base de datos, puentes Python y RoboDK). La web **no se conecta directamente** a PostgreSQL: toda la persistencia se delega al script `mqtt_bridge.py`, que escucha los mensajes publicados desde el navegador.

3.2.1. Diseño e implementación

La aplicación sigue una arquitectura **estática de tres capas lógicas**, separando contenido, presentación y comportamiento:

- **HTML** (`FABRICA DE BOMBONES.html`): define la estructura de la página como una *Single Page Application* (SPA). Las distintas zonas (inicio, servicios, detalle de pedido, cesta, mi cuenta, contacto) son secciones del mismo documento, visibles u ocultas según la navegación del usuario.
- **CSS** (`css/estilos.css`): concentra el diseño visual. Se ha empleado una paleta coherente con la identidad de la marca (marrón `#5c3a21`, dorado `#c48a5c`, fondo crema `#fef8f2`) y tipografías *Cormorant Garamond* (títulos) y *Montserrat* (texto). El layout utiliza *flexbox* y *grid* para adaptar catálogo, carrito y formularios.
- **JavaScript** (`js/app.js`): concentra toda la lógica de negocio del cliente: catálogo, carrito, validaciones, gestión de cuenta, conexión MQTT y envío de pedidos.

El diseño funcional se organiza en cuatro bloques principales:

1. **Catálogo y servicios:** visualización de los tres sabores y acceso a pedido industrial o personalizado.
2. **Mi cuenta:** registro de datos del cliente (autónomo o empresa), direcciones y métodos de pago.
3. **Cesta:** resumen del pedido, selección de empresa de reparto y confirmación.
4. **Panel MQTT:** indicador de conexión al broker y canal de comunicación con el backend.

3.2.2. Por qué se utiliza JavaScript

Se ha optado por JavaScript en el navegador por las siguientes razones:

- **Comunicación MQTT nativa en cliente:** la librería `paho-mqtt` permite conectar la web directamente al broker EMQX Cloud mediante WebSocket (`wss://`), sin necesidad de un servidor web intermedio (PHP, Flask, etc.).
- **Interactividad inmediata:** validación de formularios, cálculo de precios, límites de cajas (máximo 4 cajas grandes en carrito) y actualización dinámica del carrito sin recargar la página.
- **Persistencia local:** `sessionStorage` guarda los datos de cuenta durante la sesión; `localStorage` mantiene el carrito aunque se refresque el navegador.
- **Simplicidad de despliegue:** al ser una SPA estática, basta con abrir el archivo HTML en el navegador; no requiere instalación de servidor de aplicaciones.
- **Integración con el resto del proyecto:** el JSON publicado por JavaScript está diseñado para ser interpretado directamente por `mqtt_bridge.py`, manteniendo un contrato de datos común entre frontend y backend.

3.2.3. Partes relevantes del código JavaScript

Catálogo de productos

Los sabores se definen como un array de objetos en memoria. Cada elemento incluye identificador, nombre, precio unitario del bombón, imagen, peso y descripción:

```
const todosSabores = [  
  { id: 1, nombre: "Chocolate Negro 70%", precio: 0.75, ... },  
  { id: 2, nombre: "Caramelo Salado", precio: 0.80, ... },  
  { id: 3, nombre: "Avellana Praline", precio: 0.85, ... }  
];
```

Este catálogo alimenta tanto el pedido industrial (cajas grandes por sabor, mínimo 2 por sabor, descuento del 12 %) como el personalizado (combinación libre de cajitas dentro de cada caja grande).

Gestión de la cuenta del cliente

Los datos de *Mi cuenta* se almacenan en el objeto `datosCliente` y, al guardar, en `sessionStorage`:

```
let datosCliente = {  
  tipo: 'autonomo', cif: '', nombreApellidos: '', telefono: '',  
  email: '', empresa: '', departamento: '', tarjetas: [],  
  direccionEnvio: '', codigoPostal: '',  
  direccionFiscal: '', codigoPostalFiscal: '',  
  iban: '', titularCuenta: '', concepto: '', idCliente: null  
};
```

La función `armarClienteMqtt()` transforma estos datos al JSON que espera el backend. Según el tipo de cliente:

- **Autónomo:** envía lista de tarjetas (`tarjetas[]`) con número enmascarado, CVV y fecha de caducidad; una de ellas es la principal.
- **Empresa:** envía IBAN, titular y concepto de pago por transferencia.

Al pulsar guardar en el formulario de cuenta, la web publica:

```
publicarMqtt('tienda/cuenta/actualizada', armarClienteMqtt());
```

Carrito de compra

El carrito es un array (`carrito[]`) persistido en `localStorage`. Cada ítem puede ser de tipo **industrial** o **personalizado** e incluye precio total, descripción y detalle de sabores/cajas. La función `totalCajasEnCarrito()` impone la restricción de negocio de **máximo 4 cajas grandes** sumando industrial y personalizado.

Conexión MQTT

La conexión se establece al broker EMQX Cloud mediante WebSocket:

```
function conectarMqtt() {
  const brokerUrl = 'wss://kf12786f.ala.us-east-1.emqxsl.com:8084/mqtt';
  clienteMqtt = mqtt.connect(brokerUrl, {
    username: 'web_choco',
    password: '...',
    clientId: 'web_' + Math.random().toString(16).substr(2, 8)
  });
  // Suscripciones al conectar:
  // tienda/pedido/confirmado, tienda/stock/alerta,
  // tienda/empresas_reparto/lista, tienda/cuenta/guardada
}
```

La función `publicarMqtt(topico, mensaje)` centraliza todas las publicaciones. Si MQTT no está conectado, el mensaje no se envía y se muestra aviso en consola.

Flujo de confirmación de pedido

Cuando el usuario confirma el pedido desde la cesta, `enviarPedido()` valida que existan datos de cuenta, método de pago y empresa de reparto seleccionada. A continuación publica:

```
publicarMqtt('tienda/pedido/nuevo', {
  cliente: armarClienteMqtt(),
  idEmpresaReparto: parseInt(idReparto, 10),
  productos: carrito,
  total: total,
  nota: document.getElementById('notaPedido').value
});
```

La web permanece suscrita a `tienda/pedido/confirmado` para mostrar el número de pedido, el `idCliente` y el **número de combinación** (`numeroCombi`) asignado por el backend. Si no hay stock, recibe `tienda/stock/alerta`.

Empresas de reparto

Al entrar en la cesta, la web solicita la lista de transportistas:

```
publicarMqtt('tienda/empresas_reparto/solicitar', { solicitar: true });
```

El backend responde en `tienda/empresas_reparto/lista` y se rellena el desplegable de selección. Si MQTT no responde, se usa una lista local de respaldo (`EMPRESAS_REPARTO_FALLBACK`).

3.2.4. Relación web – MQTT – base de datos

La web **nunca escribe directamente** en PostgreSQL. El flujo es:

Web (JavaScript) → MQTT (EMQX) → `mqtt_bridge.py` → PostgreSQL

El script `mqtt_bridge.py` escucha los topics configurados en el archivo `.env` y persiste la información. La Tabla 3.2 resume qué acción de la web genera qué almacenamiento.

Cuadro 3.2: Correspondencia entre acciones de la web y tablas de PostgreSQL

Acción en la web	Topic MQTT	Proceso en <code>mqtt_bridge.py</code>	Tablas afectadas
Guardar Mi cuenta	<code>tienda/cuenta/actualizada</code>	<code>guardar_cliente_cuenta()</code>	<code>clientes</code> , <code>cuentas_bancarias</code>
Confirmar pedido	<code>tienda/pedido/nuevo</code>	<code>guardar_pedido()</code>	<code>pedidos</code> , <code>lineas_pedido</code> , <code>envios</code> , <code>pedidos_combinacion</code> , <code>stock_sabores</code>
Solicitar repartidores	<code>tienda/empresas</code>	<code>publicar_empresas_reparto()</code>	Lectura de <code>empresas_reparto</code>

3.2.5. Datos de cliente guardados en base de datos

Al guardar *Mi cuenta*, el backend inserta o actualiza un registro en `public.clientes` con:

- **Identificación:** CIF/NIF, nombre, teléfono, correo electrónico.
- **Tipo:** autónomo o empresa; en empresas, también nombre de empresa, departamento y forma jurídica (SL/SA).
- **Direcciones:** dirección de envío, código postal, dirección fiscal y código postal fiscal.

Los datos de pago se guardan en `public.cuentas_bancarias`:

- **Autónomo:** una o varias tarjetas (`tipo_metodo = 'tarjeta'`), con número, CVV, fecha de caducidad y marca de tarjeta principal.
- **Empresa:** cuenta bancaria por transferencia (`tipo_metodo = 'transferencia'`), con IBAN, titular y concepto.

Tras guardar, el backend publica `tienda/cuenta/guardada` con el `idCliente` generado, que la web almacena en `datosCliente.idCliente` para mostrarlo en el resumen del pedido.

3.2.6. Datos de pedido guardados en base de datos

Cuando el usuario confirma un pedido, `mqtt_bridge.py` ejecuta una **transacción** que:

1. Crea o actualiza el cliente (misma lógica que Mi cuenta).
2. Comprueba **stock disponible** en `stock_sabores`. Si falta material, responde con `tienda/stock/alerta` y no graba el pedido.
3. Inserta la cabecera en `public.pedidos`: tipo (`industrial` o `personalizado`), número de cajas grandes, precio total, estado `pendiente` y observaciones.
4. Crea el registro de envío en `public.envios`, asociado a la **empresa de reparto** elegida en la web.
5. Inserta las líneas del pedido en `public.lineas_pedido` (sabor y cantidad de paquetes/cajitas).
6. Descuenta stock en `stock_sabores`.
7. Si el pedido es personalizado, calcula y guarda el **número de combinación** en `public.pedidos_combinacion` (campo `numero_combi`), necesario para activar la secuencia correcta en RoboDK.

Finalmente, el backend publica `tienda/pedido/confirmado` con un resumen completo (ID pedido, ID cliente, tipo, direcciones, empresa reparto, precio, `numeroCombi`) que la web muestra en pantalla mediante `mostrarConfirmacionPedido()`.

3.2.7. Resumen del flujo completo desde la web

1. El usuario completa *Mi cuenta* → MQTT `tienda/cuenta/actualizada` → BD (`clientes`, `cuentas_bancarias`).
2. Configura pedido industrial o personalizado → se añade al carrito (local).
3. En la cesta, selecciona empresa de reparto y confirma → MQTT `tienda/pedido/nuevo` → BD (pedido completo + combinación).
4. El backend responde con confirmación y `numeroCombi` → `mqtt_station_relay.py` lo reenvía a RoboDK.

Así, la página web actúa como **origen del flujo de información**: captura la intención del cliente y la traduce a mensajes MQTT estructurados, mientras que la persistencia, la lógica de stock y la activación de la planta simulada quedan en capas posteriores del sistema.

3.2.8. Explicación detallada del `mqttbridge`

El script `mqtt_bridge.py` es el **backend central** del proyecto. Su función es escuchar los mensajes JSON que publica la página web en el broker MQTT (EMQX Cloud), traducirlos a operaciones SQL sobre PostgreSQL y, cuando corresponde, publicar respuestas hacia la web, la logística y RoboDK. La web **no accede directamente** a la base de datos: todo pasa por este puente.

Rol en la arquitectura

Página web (JavaScript) → MQTT → `mqtt_bridge.py` → PostgreSQL
↓
Respuestas MQTT: web, logística, fábrica y estación RoboDK

Al arrancar, el script:

1. Conecta a PostgreSQL y asegura tablas/columnas necesarias.
2. Se conecta al broker MQTT por WebSocket seguro (WSS).
3. Se suscribe a tres topics de entrada.
4. Procesa cada mensaje recibido en la función `on_message()`.

Configuración y dependencias

Al inicio del fichero se cargan las variables de entorno desde `.env` mediante `python-dotenv`:

```
load_dotenv()

MQTT_URL = os.getenv("MQTT_URL", "")
MQTT_TOPIC = os.getenv("MQTT_TOPIC", "tienda/pedido/nuevo")
MQTT_TOPIC_CUENTA = os.getenv("MQTT_TOPIC_CUENTA", "tienda/cuenta/actualizada")
PGHOST = os.getenv("PGHOST", "localhost")
PGDATABASE = os.getenv("PGDATABASE", "postgres")
```

Para qué sirve: separar credenciales y topics del código fuente. En producción basta con editar `.env` sin tocar el script.

Las librerías principales son:

- **pg8000:** conexión a PostgreSQL (driver puro Python).
- **paho-mqtt:** cliente MQTT con soporte WebSocket y TLS.
- **certifi:** certificados CA para conexión segura WSS.

Constantes de sabores y combinaciones

El backend define un diccionario `SABOR_ALIAS` que traduce los nombres largos de la web ("*Chocolate Negro 70 %*") a tres sabores canónicos: `chocolate`, `caramelo` y `avellana`. Esto es necesario porque la web usa nombres comerciales, pero la BD y RoboDK trabajan con identificadores fijos.

```
SABORES_ORDENADOS = ("chocolate", "caramelo", "avellana")
STOCK_ID_POR_CANON = {"chocolate": 1, "caramelo": 2, "avellana": 3}
SABOR_A_CODIGO = {"chocolate": "CHO", "caramelo": "CAR", "avellana": "AVE"}
```

El diccionario `RUTA_A_NUMERO` contiene las **34 combinaciones** del árbol de producción (alineado con `tablaHash` del PR2). Por ejemplo, la ruta `CHO-CAR` corresponde al número 5. Al iniciar el script se construyen dos mapas inversos:

- `COMBINACION_A_NUMERO`: tupla de sabores → entero.

- `NUMERO_A_COMBINACION`: entero $\rightarrow$ tupla de sabores.

Estos números son los que después recibe RoboDK para ejecutar la secuencia correcta en la estación de demostración.

3.2.9. Parte 1: utilidades de texto y tipo de pedido: `MQTTbridge`

`normalizar_nombre_sabor()`

Elimina acentos y pasa el texto a minúsculas para comparar nombres de sabores de forma uniforme, independientemente de cómo los escriba la web.

`tipo_pedido_desde_carrito()`

Recorre el array `productos` del JSON. Si alguna línea tiene `tipo == "industrial"`, el pedido se clasifica como **industrial**; en caso contrario, **personalizado**.

`_extraer_cajas_por_sabor_industrial()`

Para pedidos industriales, suma las cajas grandes por sabor a partir del campo `detalles` de cada ítem del carrito. Devuelve un diccionario `{chocolate: N, caramelo: M, avellana: K}`.

3.2.10. Parte 2: cálculo del número de combinación ;`mqttbridge`

`obtener_numero_combinacion()`

Función clave para la integración con RoboDK. Solo actúa en pedidos **industriales** con entre 1 y 4 cajas grandes (límite impuesto por la web):

```
def obtener_numero_combinacion(productos, tipo_pedido):
    if tipo_pedido != "industrial":
        return None
    cajas_por_sabor = _extraer_cajas_por_sabor_industrial(productos)
    total_cajas = sum(cajas_por_sabor.values())
    if total_cajas <= 0 or total_cajas > 4:
        return None
    combo = []
    for sabor in SABORES_ORDENADOS:
        combo.extend([sabor] * cajas_por_sabor[sabor])
    numero = COMBINACION_A_NUMERO.get(tuple(combo))
    return {"numero": numero, "ruta": combo_a_ruta(combo), ...}
```

Ejemplo: un pedido industrial con 2 cajas de chocolate y 1 de caramelo genera la tupla `(chocolate, chocolate, caramelo)`, la ruta `CHO-CHO-CAR` y el `numeroCombi = 11`.

Si el pedido es personalizado, esta función devuelve `None` y no se publica número de combinación hacia RoboDK.

3.2.11. Parte 3: preparación del esquema de base de datos

Al arrancar, `main()` ejecuta tres funciones de migración automática:

`asegurar_tablas_combinaciones()`

Crea (si no existen) las tablas `arbol_combinaciones` y `pedidos_combinacion`, e inserta las 34 combinaciones del árbol. La tabla `pedidos_combinacion` relaciona cada pedido con su `numero_combi`.

`asegurar_columnas_cliente()`

Añade a `clientes` las columnas de dirección de envío, código postal, dirección fiscal y forma jurídica, necesarias para el formulario *Mi cuenta* de la web.

`asegurar_esquema_cuentas_bancarias()`

Crea la tabla `cuentas_bancarias` con soporte dual:

- **Autónomo:** tarjetas (`tipo_metodo = 'tarjeta'`).
- **Empresa:** transferencia (`tipo_metodo = 'transferencia'`, con IBAN).

3.2.12. Parte 4: procesamiento del carrito y stock

`extraer_lineas_pedido()`

Convierte el carrito de la web en líneas unificadas por sabor, en unidades de `stock_paquetes`:

- **Industrial:** cada caja grande = 48 cajitas → se multiplica por 48.
- **Personalizado:** las cantidades ya son cajitas → factor 1.

`get_or_create_sabor()`

Resuelve el `id_sabor` en BD. Primero intenta mapear por alias (`SABOR_ALIAS`); si no encuentra el sabor, lo inserta en `sabores` y crea su fila en `stock_sabores`.

`validar_stock_disponible()`

Antes de grabar un pedido, comprueba que `stock_paquetes` $\geq$ cantidad pedida para cada sabor. Si falta stock, devuelve una lista de `faltantes` con sabor, cantidad pedida y stock actual.

`descontar_stock()`

Dentro de la misma transacción SQL, resta las cantidades pedidas de `stock_sabores` y actualiza `fecha_actualizacion`.

3.2.13. Parte 5: gestión del cliente (*Mi cuenta*)

`normalizar_cliente_web()`

Adapta el JSON de la web a un formato interno uniforme. Acepta variantes de nombres de campos (`nombre` / `nombreApellidos`, `email` / `correo`, etc.) y construye la dirección completa concatenando envío y código postal.

`extraer_tarjetas_autonomo()`

Extrae la lista de tarjetas del autónomo. Soporta el formato nuevo (`tarjetas[]`) y el legacy (un solo bloque `pago`).

`get_or_create_cliente()`

Busca el cliente por CIF en `clientes`. Si existe, hace `UPDATE`; si no, `INSERT`. Si la web no envía CIF, genera uno sintético `WEB-XXXXXXXXXXXX` mediante hash MD5 de nombre, teléfono y dirección.

`guardar_cuenta_bancaria()`

Persiste los datos de pago en `cuentas_bancarias`:

- **Empresa:** upsert de IBAN, titular y concepto (`tipo_metodo = 'transferencia'`).
- **Autónomo:** upsert de cada tarjeta; desactiva las que ya no están en la lista; marca la principal con `es_principal = TRUE`.

`guardar_cliente_cuenta()`

Función invocada al recibir `tienda/cuenta/actualizada`. Abre conexión, ejecuta `get_or_create_cliente() + guardar_cuenta_bancaria()`, hace `commit` y devuelve `{ok: True, idCliente: N}`.

3.2.14. Parte 6: guardado completo de un pedido

`guardar_pedido()`

Función central del script. Procesa el JSON de `tienda/pedido/nuevo` dentro de una **transacción SQL** (`autocommit = False`):

1. **Cliente:** `get_or_create_cliente() + guardar_cuenta_bancaria()`.
2. **Líneas:** `extraer_lineas_pedido()` convierte el carrito a unidades de stock.
3. **Validación de stock:** si hay faltantes, hace `rollback` y devuelve `{ok: False, motivo: "stock"}`.
4. **Cabecera:** inserta en `pedidos` (tipo, cajas, precio, estado `pendiente`).
5. **Envío:** `crear_envio_inicial()` inserta en `envios` con la empresa de reparto elegida en la web.
6. **Stock:** `descontar_stock()` resta material disponible.
7. **Detalle:** inserta cada sabor en `lineas_pedido`.
8. **Combinación:** si es industrial, inserta en `pedidos_combinacion` el `numero_combi` calculado.
9. **Commit** y devuelve resumen con `idPedido`, `idCliente`, `envio` y `combinacion`.

```
def guardar_pedido(payload):
    conn.autocommit = False
    # ...
    faltantes = validar_stock_disponible(cur, lineas)
    if faltantes:
        conn.rollback()
        return {"ok": False, "motivo": "stock", "faltantes": faltantes}
    # INSERT pedidos, envios, lineas_pedido, pedidos_combinacion
    conn.commit()
    return {"ok": True, "idPedido": id_pedido, "combinacion": combinacion, ...}
```

3.2.15. Parte 7: conexión MQTT

`parse_mqtt_url()`

Analiza la URL del broker (`wss://host:8084/mqtt`) y devuelve host, puerto, path, si usa WebSocket y si requiere TLS.

`on_connect()`

Callback al conectarse al broker. Se suscribe a los tres topics de entrada y publica la lista de empresas de reparto:

```
def on_connect(client, userdata, flags, reason_code, properties=None):
    client.subscribe(MQTT_TOPIC, qos=1)                # tienda/pedido/nuevo
    client.subscribe(MQTT_TOPIC_CUENTA, qos=1)         # tienda/cuenta/
    actualizada
    client.subscribe(MQTT_TOPIC_EMPRESAS_SOLICITAR, qos=1)
    publicar_empresas_reparto(client)
```

`on_message()`

Callback principal: enruta cada mensaje según el topic recibido.

Topic tienda/cuenta/actualizada

1. Llama a `guardar_cliente_cuenta(payload)`.
2. Si OK, publica `tienda/cuenta/guardada` con el `idCliente` generado (la web lo guarda en `sessionStorage`).

Topic tienda/empresas_reparto/solicitar

1. Lee `empresas_reparto` de PostgreSQL.
2. Publica `tienda/empresas_reparto/lista` con `retain=True` para que la web reciba la lista aunque se conecte después.

Topic tienda/pedido/nuevo

1. Llama a `guardar_pedido(payload)`.
2. **Si falta stock:** publica `tienda/stock/alerta` hacia la web y limpia el topic de estación RoboDK con `rechazado_sin_stock`.
3. **Si OK:** publica en cadena:
 - `tienda/pedido/confirmado` → web (resumen completo del pedido).
 - `logistica/pedido/{id}` → módulo logístico.
 - `fabrica/esp32/pedido_guardado` → aviso a planta/ESP32.
 - `numero_combi/asignado` → número de combinación (retain).
 - `giirob/pr2/station/demo/status` → entero del `numeroCombi` para `mqtt_station_relay.py` y RoboDK.

```
if combinacion:
    numero_combi = combinacion["numero"]
    client.publish(NUMERO_COMBI_TOPIC, json.dumps({...}), qos=1, retain=True)
    client.publish(STATION_STATUS_TOPIC, str(numero_combi), qos=1, retain=True)
```

El flag `retain=True` garantiza que RoboDK reciba el último número de combinación aunque el relay se conecte después del pedido.

3.2.16. Parte 8: función `main()` – arranque del servicio

```
def main():
    comprobar_variables_mqtt()          # valida .env
    conn = pg8000.connect(...)
    asegurar_tablas_combinaciones(conn) # migraciones BD
    asegurar_columnas_cliente(conn)
    asegurar_esquema_cuentas_bancarias(conn)
    conn.close()

    host, port, path, use_ws, use_tls = parse_mqtt_url(MQTT_URL)
    cli = mqtt_client.Client(transport="websockets" if use_ws else "tcp")
    if use_tls:
        cli.tls_set(ca_certs=certifi.where())
    cli.username_pw_set(MQTT_USERNAME, MQTT_PASSWORD)
    cli.on_connect = on_connect
    cli.on_message = on_message
    cli.connect(host, port, keepalive=60)
    cli.loop_forever()                  # bucle infinito escuchando MQTT
```

3.2.17. Tabla resumen de topics MQTT

Cuadro 3.3: Topics MQTT gestionados por `mqtt_bridge.py`

Topic	Dir.	Cuándo	Contenido principal
tienda/cuenta/actualizada	Entrada	Usuario guarda Mi cuenta	JSON cliente (CIF, direcciones, pago)
tienda/cuenta/guardada	Salida	Tras guardar cuenta	{ok, idCliente}
tienda/empresas_reparto/solicitar	Entrada	Web entra en cesta	{solicitar: true}
tienda/empresas_reparto/lista	Salida	Respuesta a solicitud	{empresas: [{id, nombre}, ...]}
tienda/pedido/ nuevo	Entrada	Usuario confirma pedido	JSON con cliente, productos, total, idEmpresaReparto
tienda/pedido/confirmado	Salida	Pedido guardado OK	Resumen completo + numeroCombi
tienda/stock/ alerta	Salida	Stock insuficiente	Lista de sabores faltantes
numero_combi/ asignado	Salida	Pedido industrial OK	{numeroCombi, ruta, sabores} (retain)
giirob/pr2/station/demo/status	Salida	Pedido industrial OK	Entero numeroCombi (retain)
logistica/pedido/{id}	Salida	Pedido guardado OK	Datos de envío y reparto
fabrica/esp32/pedido_guardado	Salida	Pedido guardado OK	Aviso de producción (idPedido, tipo)

3.2.18. Tabla resumen: qué se guarda en PostgreSQL

Cuadro 3.4: Tablas de PostgreSQL afectadas por `mqtt_bridge.py`

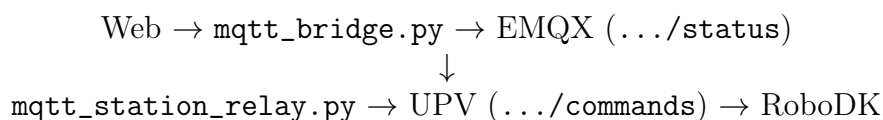
Tabla	Cuándo se escribe	Campos principales
<code>clientes</code>	Mi cuenta o nuevo pedido	CIF, nombre, teléfono, correo, tipo, direcciones, forma jurídica
<code>cuentas_bancarias</code>	Mi cuenta o nuevo pedido	Tarjetas (autónomo) o IBAN (empresa)
<code>pedidos</code>	Confirmar pedido	<code>id_cliente</code> , tipo, cajas, precio, estado, observaciones
<code>lineas_pedido</code>	Confirmar pedido	<code>id_pedido</code> , <code>id_sabor</code> , <code>cantidad_paquetes</code>
<code>envios</code>	Confirmar pedido	<code>id_pedido</code> , <code>id_empresa_reparto</code> , estado pendiente
<code>pedidos_combinacion</code>	Pedido industrial OK	<code>id_pedido</code> , <code>numero_combi</code> , <code>tamano</code> , <code>sabores_json</code>
<code>stock_sabores</code>	Confirmar pedido	Descuento de stock_paquetes
<code>arbol_combinaciones</code>	Arranque del script	Las 34 combinaciones CHO/-CAR/AVE (solo inicialización)

En conjunto, `mqtt_bridge.py` actúa como **traductor y validador** entre la interfaz web y la persistencia en base de datos, garantizando integridad transaccional, control de stock y coordinación del sistema automatizado.

3.3. Puente MQTT_station_relay

El script `mqtt_station_relay.py` actúa como **puente entre dos brokers MQTT distintos**. La web y `mqtt_bridge.py` trabajan sobre EMQX Cloud (Chocolatillo), mientras que RoboDK está conectado al broker de la UPV (`mqtt.dsic.upv.es`). Este script traduce el `numeroCombi` de un broker al otro para que la estación simulada ejecute la secuencia correcta.

3.3.1. Rol en el sistema



El puente **no accede a la base de datos**. Solo escucha un topic en EMQX y publica en otro en el broker UPV.

3.3.2. Configuración (.env)

Los topics y credenciales se leen del archivo `.env`:

```
STATION_STATUS_TOPIC=giirob/pr2/station/demo/status
STATION_COMMANDS_TOPIC=giirob/pr2/station/demo/commands
PROYECTOS_MQTT_HOST=mqtt.dsic.upv.es
PROYECTOS_MQTT_PASSWORD=... # obligatorio para arrancar
```

3.3.3. Doble conexión MQTT

Al iniciar, el script crea **dos clientes MQTT** en paralelo:

```
proyectos = crear_cliente("", PROYECTOS_MQTT_USERNAME,
                          PROYECTOS_MQTT_PASSWORD, "relay_dst",
                          host=PROYECTOS_MQTT_HOST, port=PROYECTOS_MQTT_PORT)
choco = crear_cliente(MQTT_URL, MQTT_USERNAME, MQTT_PASSWORD, "relay_src")
```

- **choco** (origen): se conecta a EMQX por WSS y se suscribe a `STATION_STATUS_TOPIC`.
- **proyectos** (destino): se conecta a `mqtt.dsic.upv.es` por TCP y publica en `STATION_COMMANDS_TOPIC`.

3.3.4. Extracción del número de combinación

Cuando `mqtt_bridge.py` confirma un pedido industrial, publica el entero del `numeroCombi` (por ejemplo, "11") en el topic de `status`. El relay lo interpreta con `extraer_numero_combinacion()`:

```
def extraer_numero_combinacion(payload):
    texto = payload.decode("utf-8").strip()
    if texto.lower().startswith("ready"):
        return None # mensaje de RoboDK, se ignora
    if re.fullmatch(r"\d+", texto):
        n = int(texto)
        if 1 <= n <= 99:
            return n
    return None
```

También ignora mensajes de pedido rechazado por stock (`rechazado_sin_stock`), evitando que RoboDK ejecute una combinación inválida.

3.3.5. Reenvío hacia RoboDK

Al recibir un número válido, el callback `on_message_choco` lo publica en el broker UPV:

```
def on_message_choco(client, userdata, msg):
    numero = extraer_numero_combinacion(msg.payload)
    if numero is None:
        return
    plano, json_b = payloads_para_proyectos(numero)
    publicar(proyectos, STATION_COMMANDS_TOPIC, plano) # ej. "11"
```

RoboDK recibe el número en texto plano en `giirob/pr2/station/demo/commands` y lanza la secuencia asociada a esa combinación (transporte, Pick and Place, paletizado).

3.3.6. Ejecución

Desde la carpeta del proyecto, con `mqtt_bridge.py` ya en marcha:

```
python mqtt_station_relay.py
```

Salida esperada:

```
Proyectos OK -> commands: giirob/pr2/station/demo/commands
Chocolatillo OK -> status: giirob/pr2/station/demo/status
Activo. Haz un pedido en la web con relay + mqtt_bridge corriendo.
```

3.4. Sistemas de seguridad con ESP32

Además del flujo principal de pedidos (web → `mqtt_bridge.py` → RoboDK), el proyecto incluye una **segunda planta virtual** orientada a demostrar funciones de **seguridad industrial** y la interacción con sensores externos. Un microcontrolador **ESP32-S3** publica comandos MQTT que son recibidos por RoboDK a través del broker de la UPV.

El funcionamiento global es el siguiente:

```
ESP32 (botones + ultrasonido) → broker EMQX
    ↓ mqtt_industrial_bridge.py (PC)
broker UPV (mqtt.dsic.upv.es) → MQTT_listener.py (RoboDK)
```

El ESP32 **no habla directamente con RoboDK**. Publica en MQTT; el script `mqtt_industrial_bridge.py` reenvía los mensajes al broker UPV, donde el listener de RoboDK los interpreta y actúa sobre los robots simulados.

Los cuatro comandos implementados son:

- **PER** — interrumpe el paletizado industrial y conmuta al proceso personalizado.
- **STOP** — parada de emergencia del robot de cajas.
- **SLOW** — reduce la velocidad de los robots colaborativos C1/C2; robots colaborativos. (proximidad detectada).
- **NORMAL** — restaura la velocidad nominal de C1/C2; robots colaborativos.

3.4.1. Elementos físicos usados

El hardware montado para la demostración consta de tres elementos conectados al ESP32:

Los pulsadores están configurados con `INPUT_PULLUP`: en reposo leen nivel alto y, al pulsarse, nivel bajo. Así se detecta el flanco de pulsación sin rebotes software complejos.

El HC-SR04 mide la distancia mediante el tiempo de vuelo del eco ultrasónico. Si la distancia cae por debajo de **7 cm**, se publica **SLOW**; si supera **9 cm** (histéresis), se publica **NORMAL**. Esto simula el comportamiento exigido por normativas de robots colaborativos (ISO/TS 15066) ante presencia de personas u objetos en la zona de trabajo.

Cuadro 3.5: Componentes del sistema de seguridad ESP32

Componente	Conexión	Comando MQTT	Función
Pulsador membrana 1	GPIO 4 → GND	PER	Cambio a pedido personalizado
Pulsador membrana 2	GPIO 5 → GND	STOP	Parada de emergencia
Sensor ultrasónico HC-SR04	TRIG=GPIO 6, ECHO=GPIO 7	SLOW / NORMAL	Control de velocidad por proximidad

3.4.2. Código Arduino

Se programó con **Arduino IDE**, utilizando la librería **PubSubClient** para MQTT y las APIs nativas de **WiFi** del ESP32.

Configuración de red (secrets.h)

Las credenciales **no van en el .ino**. Se copia la plantilla `secrets.h.example` a `secrets.h` y se rellenan los datos de red:

```
#define WIFI_SSID "UPV-PSK"
#define WIFI_PASSWORD "giirob-pr2-2023"

#define MQTT_HOST "kf12786f.ala.us-east-1.emqxsl.com"
#define MQTT_PORT 8883
#define MQTT_USER "web_choco"
#define MQTT_PASS "... " // misma contraseña que MQTT_PASSWORD en .
    env
#define MQTT_USE_TLS 1

#define MQTT_TOPIC_COMMANDS "giirob/pr2/station/demo/commands"
```

Importante: el ESP32 debe conectarse a la **misma red WiFi** que el ordenador donde se ejecuta `mqtt_industrial_bridge.py`. Si el PC no tiene acceso a Internet o a EMQX, el ESP32 tampoco podrá publicar. En el laboratorio se utiliza la red UPV-PSK; en casa, hay que poner el SSID y la contraseña de la red local del usuario.

Lógica principal del .ino

En `setup()` se configuran los pines, se conecta a WiFi y al broker MQTT. En `loop()` se ejecutan tres tareas de forma cíclica:

1. Mantener la conexión WiFi/MQTT activa (`conectarWifiYMqtt()`).
2. Leer los pulsadores y publicar PER o STOP solo en el flanco de pulsación (evita repeticiones).
3. Leer el ultrasonido cada 150 ms y publicar SLOW o NORMAL según el umbral.

Fragmento representativo de la publicación MQTT:

```

void publicarComando(const char* comando) {
    if (!mqtt.connected()) return;
    mqtt.publish(MQTT_TOPIC_COMMANDS, comando);
}

// En loop(): boton PER (GPIO 4)
if (p_per && !antes_per) {
    publicarComando("PER");
}

```

Tras subir el sketch, el monitor serie (115200 baudios) debe mostrar mensajes del tipo Publish [...] ->PER OK al pulsar cada botón.

Puente en el PC

Como el ESP32 publica en **EMQX Cloud** y RoboDK escucha en **mqtt.dsic.upv.es**, en el ordenador debe estar en ejecución:

```
python mqtt_industrial_bridge.py
```

Este script reenvía PER, STOP, SLOW y NORMAL del broker EMQX al mismo topic en el broker UPV.

3.4.3. MQTT_listener en RoboDK

El listener MQTT es el programa **único** que debe ejecutarse dentro de RoboDK (tecla F5). El fichero MQTT_listener.py está en choco_esp32_mqtt/ y concentra **toda** la lógica de recepción MQTT y control de robots en un solo archivo.

Por qué todo va en un solo archivo

RoboDK copia los scripts Python a una carpeta temporal (Temp) al ejecutarlos. Si MQTT_listener.py intentara importar un RobotController.py separado, **no lo encuentra** y el sistema falla. Por eso el controlador de robots está **integrado dentro del listener**: funciones como handle_message(), iniciar_industrial(), iniciar_personal() y _parar_robot() viven en el mismo fichero que la conexión MQTT.

Los programas de paletizado (Script_P&P_paletizado, P&P_per, etc.) **sí permanecen separados** en el árbol de RoboDK; el listener solo los invoca con RunProgram().

Conexión MQTT y arranque

Al iniciar, el listener se conecta al broker UPV y se suscribe al topic de comandos:

```

broker = 'mqtt.dsic.upv.es'
station_commands_topic = 'giirob/pr2/station/demo/commands'
station_status_topic = 'giirob/pr2/station/demo/status'

def on_connect(mqttdc, obj, flags, reason_code, properties=None):
    mqttdc.subscribe(station_commands_topic, 0)
    mqttdc.publish(station_status_topic, 'ready')
    iniciar_c1_paralelo()
    iniciar_c2_paralelo()

```

```

    iniciar_industrial(mqttc)

mqttc.loop_forever()

```

Al conectar, arranca automáticamente el paletizado industrial y lanza en paralelo los programas de los robots colaborativos C1 y C2 (P&P_c1, P&P_c2).

Interpretación de comandos (handle_message)

Cada mensaje recibido en `commands` se procesa en `handle_message()`:

```

def handle_message(mqttc, topic, payload):
    txt_u = str(payload).strip().upper()

    if txt_u == 'STOP':
        _parar_robot()
        mqttc.publish(TOPIC_STATUS, 'stopped')
    elif txt_u == 'PER':
        _parar_robot()
        iniciar_personal(mqttc)      # RunProgram("P&P_per")
    elif txt_u == 'SLOW':
        _set_speed_c1_c2(20.0)
    elif txt_u == 'NORMAL':
        _set_speed_c1_c2(60.0)
    elif txt_u in ('START', 'INDUSTRIAL', 'RUN'):
        iniciar_industrial(mqttc)    # RunProgram("Script_P&P_paletizado")

```

Cuadro 3.6: Comandos MQTT y acción en RoboDK

Payload	Acción en RoboDK	Respuesta en status
PER	Para industrial y ejecuta P&P_per	switch:PER
STOP	Parada de emergencia (RDK.Stop())	stopped
SLOW	Velocidad C1/C2 a 20 °/s	speed:slow
NORMAL	Velocidad C1/C2 a 60 °/s	speed:normal
START	Relanza paletizado industrial	running:industrial

Resolución de programas en el árbol de RoboDK

Al arrancar, `inicializar_programas()` busca en el árbol los programas por nombre (Script_P&P_paletizado, P&P_per, P&P_c1, P&P_c2), incluso si están dentro de carpetas como P&P_Grande/ o P&P_PER/. Si no los encuentra, imprime en consola la lista de programas disponibles para facilitar la depuración.

3.5. Programacion en RoboDK

3.5.1. Organización de las secuencias

Dentro del proyecto de RoboDK se encuentran distintas carpetas, siendo la más importante la denominada “Secuencias”. Esta carpeta se encuentra organizada en cuatro subcarpetas:

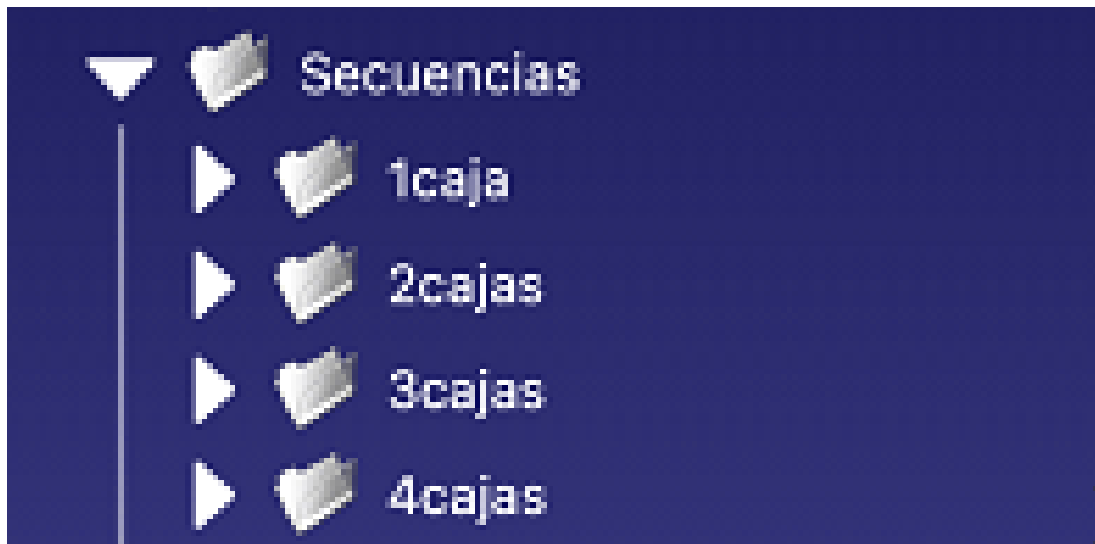


Figura 3.3: Organización de carpetas en RoboDK

- 1 Caja
- 2 Cajas
- 3 Cajas
- 4 Cajas

Esta estructura se implementó para facilitar la navegación dentro del proyecto y hacer más sencilla la visualización de las distintas simulaciones. Cada carpeta contiene secuencias correspondientes a una cantidad determinada de cajas solicitadas por el cliente.

Por ejemplo, la carpeta “1 Caja” contiene secuencias asociadas a pedidos personalizados de una única caja de un sabor específico, mientras que el robot industrial realiza simultáneamente una operación de Pick and Place relacionada con un pedido industrial generado de forma aleatoria.

En total, se desarrollaron 32 secuencias diferentes dentro de RoboDK, implementadas como programas internos de la aplicación.

3.5.2. Estructuración de los procesos

Para simplificar el desarrollo y mantenimiento de las secuencias, los distintos procesos fueron divididos en carpetas independientes según su función. Esta metodología permitió reutilizar programas ya desarrollados y construir secuencias complejas de forma modular.

Las principales carpetas utilizadas fueron:

Movimientos de cintas transportadoras: incluían los programas encargados del desplazamiento de productos y cajas a través de la planta.

Tipos de cajas: contenían las rutinas correspondientes a las diferentes configuraciones de cajas utilizadas en los pedidos personalizados y en masa.

Pick and Place: agrupaban los movimientos del robot industrial destinados a la toma y colocación de productos según la combinación solicitada.

Paletizado: incluían las secuencias necesarias para organizar y depositar las cajas terminadas sobre los pallets.

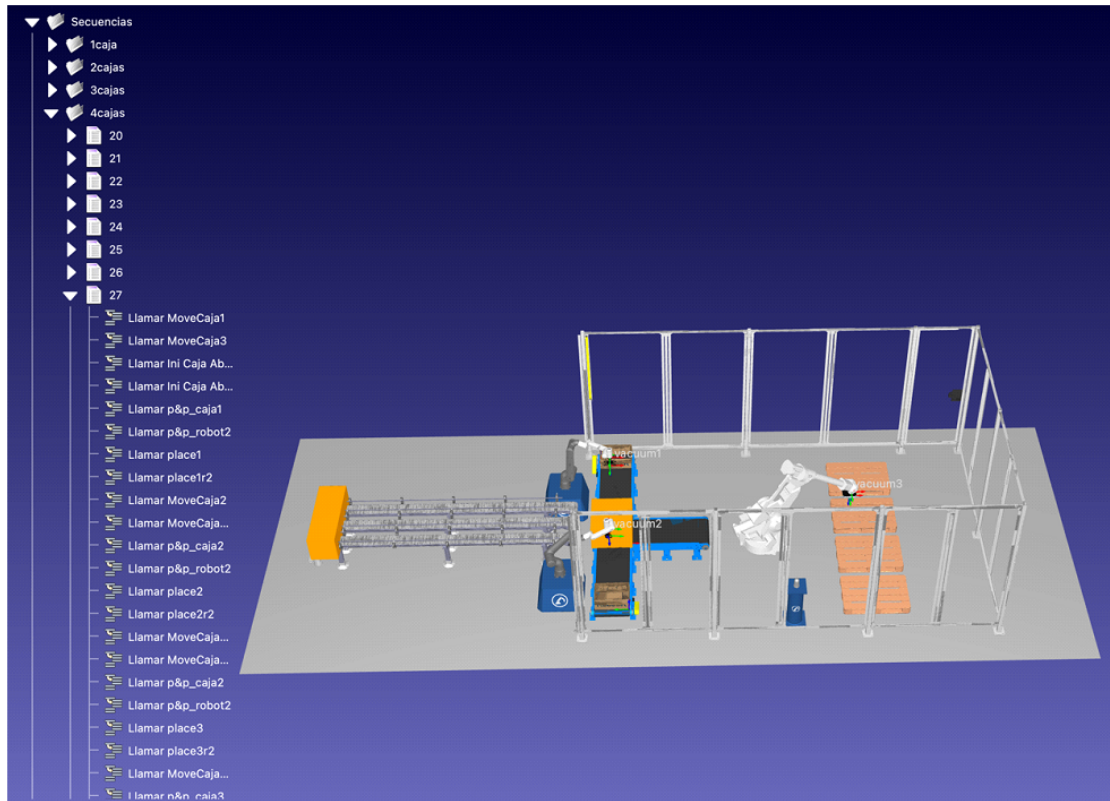


Figura 3.4: Ejemplo de secuencia en RoboDK

3.5.3. Desarrollo de las secuencias

Inicialmente se planteó la posibilidad de generar todas las secuencias mediante programación en Python. Sin embargo, debido a la gran cantidad de combinaciones y programas necesarios, el sistema terminaba sobrecargándose y no ejecutaba correctamente las simulaciones.

Por esta razón, se optó por implementar las secuencias directamente dentro de RoboDK como programas independientes. Esta solución permitió mejorar el rendimiento de la simulación y garantizar una ejecución estable de todas las combinaciones de pedidos.

3.5.4. Sistema de señalización

Además de las secuencias de movimiento, se incorporó un actuador luminoso (LED) programado dentro de RoboDK para simular el sistema de señalización utilizado durante el proceso de paletizado.

Este indicador visual permite representar tres estados diferentes del sistema:

Proceso detenido o apagado.

Proceso en funcionamiento.

Proceso finalizado, indicando que el operario puede acceder de forma segura para retirar la caja paletizada.

De esta forma, la simulación reproduce el comportamiento de un sistema industrial real, proporcionando información visual sobre el estado actual del proceso.

3.5.5. Relacion MQTT-python

Para establecer la comunicación entre el sistema de control y RoboDK se desarrollaron programas en Python utilizando un MQTT Listener. Este componente se conecta al controlador MQTT del sistema y permanece a la espera de los mensajes enviados por el resto de dispositivos.

Cuando se recibe un mensaje MQTT, el programa interpreta el número de combinación asociado al pedido y selecciona automáticamente la secuencia correspondiente dentro de RoboDK, ejecutando la simulación adecuada para cada caso. Adicionalmente, se implementó un botón de parada de emergencia (Pause Button) programado en Python. Este mecanismo permite detener inmediatamente todos los procesos en ejecución cuando es activado, garantizando la seguridad de la instalación y reproduciendo el comportamiento esperado en un entorno industrial real.

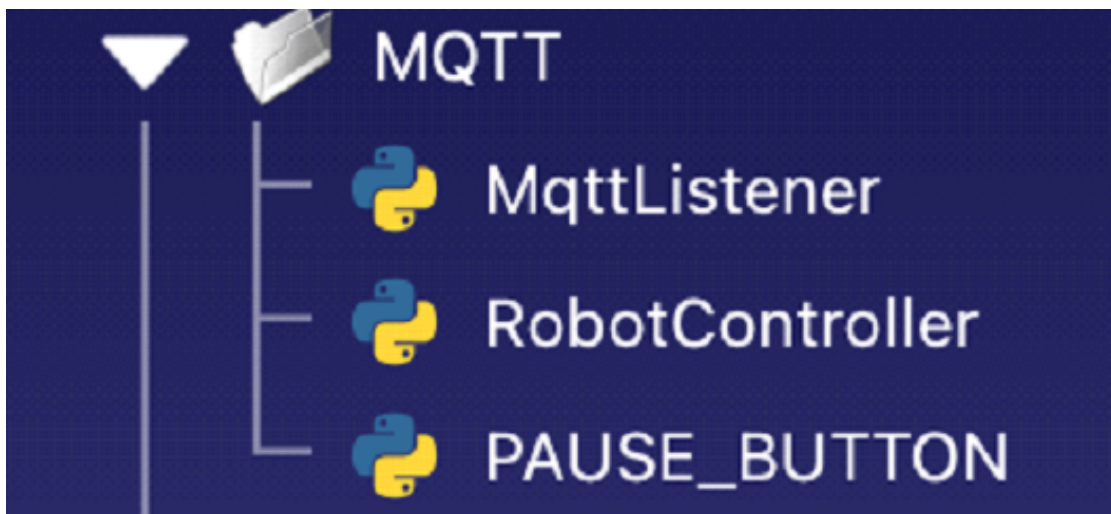


Figura 3.5: arbol programas

```
from robodk import robolink
from robodk.robolink import ITEM_TYPE_PROGRAM
import threading

RDK = robolink.Robolink()

import paho.mqtt.client as mqtt

station_commands_topic = "giirob/pr2/station/demo/commands"
station_status_topic = "giirob/pr2/station/demo/status"

def handle_message(mqttc, topic, payload):
    msg = payload.strip()

    print(" Mensaje recibido:", msg)

    if topic == station_commands_topic:

        if msg == "1":
            hilo = threading.Thread(
                target=ejecutar_programa,
                args=(mqttc, "1")
            )
            hilo.start()

        elif msg == "2":
            hilo = threading.Thread(
                target=ejecutar_programa,
                args=(mqttc, "2")
            )
            hilo.start()

        elif msg == "3":
            hilo = threading.Thread(
                target=ejecutar_programa,
                args=(mqttc, "3")
            )
```

Figura 3.6: código



```
from robolink import *
from robodk import *

RDK = Robolink()

# Detiene el robot o programa activo en la simulación
robot = RDK.Item('robot1c', ITEM_TYPE_ROBOT)
robot.Stop()

robot = RDK.Item('robot2c', ITEM_TYPE_ROBOT)
robot.Stop()

robot = RDK.Item('robot3c', ITEM_TYPE_ROBOT)
robot.Stop()

robot = RDK.Item('cinta1c', ITEM_TYPE_OBJECT)
robot.Stop()

robot = RDK.Item('cinta2c', ITEM_TYPE_OBJECT)
robot.Stop()

robot = RDK.Item('cinta3c', ITEM_TYPE_OBJECT)
robot.Stop()

robot = RDK.Item('cintag1c', ITEM_TYPE_OBJECT)
robot.Stop()

robot = RDK.Item('cintag2c', ITEM_TYPE_OBJECT)
robot.Stop()

robot = RDK.Item('cintag3c', ITEM_TYPE_OBJECT)
robot.Stop()

# Borra todos los programas o elementos del entorno
RDK.Command("Reset")
```

Figura 3.7: código

Capítulo 4

Cronograma y análisis financiero

4.1. CRONOGRAMA y gestión del proyecto

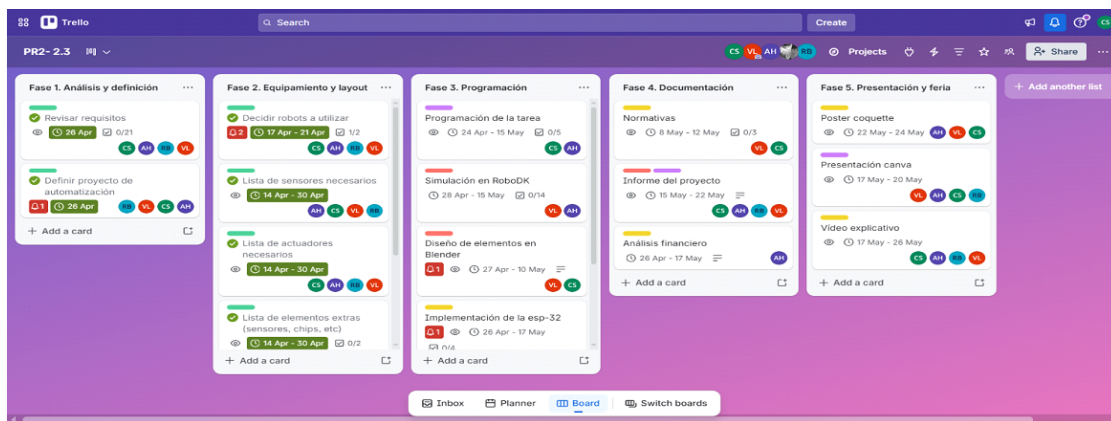


Figura 4.1: Cronograma del proyecto (Trello)

Hemos organizado el proyecto en cinco fases principales, gestionadas a través de la herramienta Trello y distribuidas entre los meses de abril y mayo.

Viéndolo más a detalle, vemos que las fases, tareas e hitos principales son los siguientes:

- Fase 1 – Análisis y definición: análisis de requisitos y definición del proyecto de automatización
- Fase 2 – Equipamiento y layout: selección de robots, sensores, actuadores y elementos adicionales necesarios para la planta
- Fase 3 – Programación: programación de las tareas del sistema, simulación en RoboDK, implementación de la ESP32-S3 y desarrollo de la base de datos
- Fase 4 – Documentación: redacción del informe del proyecto, normativas y análisis financiero
- Fase 5 – Presentación y feria: elaboración del poster, presentación de Canva y vídeo explicativo

Dividimos las responsabilidades entre los cuatro miembros del equipo, aunque no todas participamos en todas las fases del proyecto.

4.1.1. Pruebas y validación

Decidimos dividir las pruebas del funcionamiento del sistema en dos niveles:

- Simulación en RoboDK – validamos el comportamiento de todos los robots en el entorno simulado, comprobando la viabilidad de los movimientos, la ausencia de colisiones y la correcta secuencia de operaciones. Los resultados esperados incluyen la ejecución completa del ciclo de empaquetado y paletizado según un pedido de la página web sin interrupciones ni errores graves.
- Pruebas de comunicación MQTT – verificamos que hubiera un correcto flujo de mensajes entre el backend Python y la ESP32-S3 en ambos escenarios, comprobando que los topics, estructuras de datos y respuestas del sistema funcionan según lo previsto.

4.2. Costes y beneficios

Con el fin de evaluar la viabilidad económica de la propuesta de automatización, se ha realizado un análisis comparativo entre la situación manual previa y la situación automatizada, estimando la inversión inicial necesaria, los gastos operativos anuales asociados a cada escenario y los principales indicadores de rentabilidad.

Inversión inicial

La inversión total del proyecto asciende a **91.891,07 €**, desglosada en 59.391,07 € de materiales (activos no corrientes) y 32.500,00 € en instalación, configuración, formación y medidas de seguridad. La Tabla 4.1 recoge el detalle de los componentes principales.

Cuadro 4.1: Inversión inicial del proyecto de automatización

Componente	Importe (€)	Tipo
ABB IRB 4400/L30 M2000	14.500,00	Activo no corriente
ABB CRB 15000	32.000,00	Activo no corriente
VG20 Vacuum Gripper (OnRobot)	6.530,00	Activo no corriente
VG10 Vacuum Gripper (OnRobot)	4.201,00	Activo no corriente
Cinta transportadora VEVOR 1498,6×198,1 mm	256,90	Activo no corriente
Cinta transportadora VEVOR 1498,6×398,8 mm	256,90	Activo no corriente
Máquina de cerrado y empaquetado	781,38	Activo no corriente
Baliza PATLITE LR4-302WJNW-RYG	122,57	Activo no corriente
Lector de códigos Hikvision ID6000	350,90	Activo no corriente
Báscula industrial Accurex TX-Tiger	350,90	Activo no corriente
Interruptor de límite Omron D4N	40,52	Activo no corriente
Total materiales	59.391,07	
Otros gastos (instalación, configuración, formación, seguridad)	32.500,00	
Inversión total inicial	91.891,07	

Comparativa de gastos anuales

Para calcular el ahorro generado por la automatización, se han estimado los costes anuales en ambas situaciones, considerando principalmente el coste de personal (salarios brutos más seguridad social al 30 %) y las pérdidas por mermas y errores en producción. Los resultados se recogen en la Tabla 4.2.

Cuadro 4.2: Comparativa de gastos anuales antes y después de la automatización

Concepto	Manual (€)	Automatizado (€)
Número de operarios	23	4
Salario bruto anual por operario	24.317,00	24.317,00
Total salarios brutos	559.291,00	97.268,00
Costes sociales (30 %)	167.787,30	29.180,40
Subtotal personal	727.078,30	126.448,40
Tasa de mermas anual	3,00 %	1,00 %
Pérdidas por mermas	11.880,00	24.750,00
Total gastos anuales	738.958,30	277.646,80

La reducción de operarios de 23 a 4 constituye el principal factor de ahorro. Aunque la producción automatizada incrementa el volumen anual de cajas grandes procesadas (de 26.400 a 66.000 unidades), la tasa de mermas desciende del 3 % al 1 %, lo que mitiga parcialmente el incremento absoluto de pérdidas por defectos asociado al mayor volumen productivo.

Metodología de evaluación financiera

Para valorar la viabilidad económica del proyecto se han empleado cuatro indicadores: ROI, VAN, TIR, PayBack .

El payback y el ROI resultan útiles, pero no incorporan el valor temporal del dinero. Por ello, en proyectos de inversión industrial, el VAN y la TIR complementan el análisis y permiten comparar la rentabilidad del proyecto con el coste de capital de la empresa.

Supuestos del análisis dinámico (VAN y TIR)

- Horizonte de evaluación: **10 años** (vida útil estimada de la celda).
- Ahorro operativo anual bruto: **461.311,50 €** (diferencia entre escenarios manual y automatizado).
- Coste anual de mantenimiento: **5 %** de la inversión inicial (≈ 4.594 €/año).
- Flujo de caja neto anual constante: **456.717 €/año**.
- Tipo de descuento para el VAN: **8 %** (coste de oportunidad del capital).

Estos supuestos constituyen simplificaciones. Un estudio industrial completo debería incluir variaciones de demanda, inflación salarial, coste energético y valor residual de los equipos al final del horizonte.

Resultados e indicadores financieros

Ahorro neto anual. Representa la diferencia entre los gastos anuales del proceso manual y los del proceso automatizado:

$$\text{Ahorro neto anual} = 738,958,30 - 277,646,80 = 461,311,50 \text{ €}$$

Este valor cuantifica el beneficio económico directo que la empresa obtendría cada año gracias a la reducción de personal en la línea de empaquetado y paletizado, así como a la mejora en la eficiencia productiva y en el control de mermas.

Periodo de recuperación (payback). El payback indica el tiempo necesario para recuperar la inversión inicial a partir del ahorro anual generado:

$$\text{Payback (meses)} = \frac{\text{Inversión inicial}}{\text{Ahorro neto anual}/12} = \frac{91,891,07}{461,311,50/12} \approx 2,39 \text{ meses}$$

Un periodo de recuperación inferior a tres meses supone que la inversión se amortiza en menos de un trimestre, resultado especialmente favorable en el ámbito de la automatización industrial.

Retorno sobre la inversión (ROI). El ROI mide la rentabilidad de la inversión en términos porcentuales. Se emplea la formulación basada en el beneficio neto del primer ejercicio:

$$\text{ROI} = \frac{\text{Ahorro neto anual} - \text{Inversión inicial}}{\text{Inversión inicial}} \times 100 = \frac{461,311,50 - 91,891,07}{91,891,07} \times 100 \approx 402 \%$$

Un ROI del 402 % indica que, en el primer año de explotación, el beneficio neto equivale a más de cuatro veces la inversión inicial. No debe confundirse con el ROI simple ($\text{Ahorro}/\text{Inversión} \approx 502 \%$), que no resta la inversión del numerador.

Valor Actual Neto (VAN). El VAN incorpora el valor temporal del dinero descontando los flujos futuros al tipo del 8 %:

$$\text{VAN} = -91,891,07 + \sum_{t=1}^{10} \frac{456,717}{(1,08)^t} = -91,891,07 + 456,717 \times 6,710 \approx 2,972,000 \text{ €}$$

Al ser $\text{VAN} > 0$, el proyecto **crea valor** para la empresa según el tipo de descuento adoptado y resultaría admisible desde el punto de vista financiero.

Tasa Interna de Retorno (TIR). La TIR es el tipo de interés i que verifica $\text{VAN}(i) = 0$. Con los supuestos indicados, la TIR estimada supera ampliamente el 8 % de descuento (del orden de **450–500 %** anual), lo que confirma la rentabilidad del proyecto bajo el modelo adoptado. No obstante, un valor tan elevado evidencia que la relación ahorro/inversión es excepcionalmente favorable y debe interpretarse junto con las limitaciones del modelo.

Conclusiones económicas

Los cuatro indicadores analizados —payback, ROI, VAN y TIR— llegan a la misma conclusión: la automatización del empaquetado y paletizado presenta una viabilidad económica muy elevada para *Smart Chocolate Factory*.

En términos cuantitativos, una inversión inicial de 91.891,07 € genera un ahorro neto anual de 461.311,50 €, con un periodo de recuperación de apenas 2,39 meses. El VAN positivo (aproximadamente 2,97 millones de euros a diez años y tipo de descuento del 8 %) confirma que el proyecto crea valor a largo plazo, mientras que la TIR estimada supera ampliamente el coste del capital, reforzando la decisión de inversión.

No obstante, conviene señalar que estos resultados se basan en estimaciones simplificadas. En un análisis industrial real, estos indicadores deberían complementarse con un estudio de sensibilidad y con la inclusión de costes operativos adicionales.

A pesar de estas limitaciones, el conjunto del análisis financiero respalda de forma sólida la propuesta de automatización como una inversión rentable, alineada con los objetivos de mejora de productividad y reducción de costes operativos planteados en el Capítulo 2.

Capítulo 5

Normativa, regulación y seguridad

5.0.1. Normativas y estándares aplicables

El proyecto de automatización de Chocolat Supreme está sujeto a una serie de normativas europeas y españolas para regular tanto el uso de maquinaria industrial como las condiciones de trabajo y la seguridad en planta. Para este proyecto, consideramos que las principales que se nos aplicaban eran:

- Directiva de Maquinaria 2006/42/CE – establece los requisitos esenciales de seguridad y salud que debe cumplir toda maquinaria comercializada en la UE, incluyendo robots industriales y cintas transportadoras, por tanto, todos los equipos adquiridos para este proyecto deben disponer del marcado CE correspondiente
- ISO 10218-1 e ISO 1021-2 – normas internacionales que regulan los requisitos de seguridad para robots industriales, tanto para el propio robot (parte 1) como para su integración en el sistema de producción (parte 2), lo cual es directamente aplicable al ABB de paletizado.
- ISO/TS 15066 – específica para robots colaborativos, regula los límites de fuerza y velocidad permitidos en operaciones donde el robot puede interactuar con personas, lo que debemos aplicar a los ABB IRB en su zona de trabajo compartida con operarios.
- ISO 13849-1 – define los requisitos para el diseño de sistemas de control relacionados con la seguridad incluyendo paradas de emergencia y enclavamientos, debemos seguirla para el diseño del sistema de control general de la planta.
- Reglamento CE 852/2004 sobre higiene alimentaria – al ser una empresa del sector alimentario, todos los equipos en contacto directo o indirecto con el producto deben cumplir los requisitos de higiene establecidos, por lo que, aunque los robots nunca toquen los bombones directamente, sus actuadores (grippers de vacío) y las cintas transportadoras deben estar conforme a la normativa
- Ley 31/1995 de Prevención de Riesgos Laborales (LPRL) – marco legal español que obliga a la empresa a evaluar y controlar los riesgos derivados de la introducción de nueva maquinaria, así como a informar y formar a los trabajadores afectados.
- Real Decreto 1215/1997 – establece las disposiciones mínimas de seguridad y salud para la utilización de equipos de trabajo, incluyendo maquinaria automatizada.

5.0.2. Consideraciones de seguridad en planta

La convivencia de robots industriales y operarios en el mismo espacio de trabajo requiere una serie de medidas de seguridad específica, las cuales hemos implementado y se aprecian en la foto del diseño de la planta:

- Vallado y delimitación de zonas – la zona de operación del robot industrial de paletizado debe estar físicamente delimitada mediante vallas de seguridad o barreras de luz, impidiendo el acceso humano durante su funcionamiento
- Paradas de emergencia – el sistema debe disponer de setas de parada de emergencia accesibles en distintos puntos de la planta, conectadas al sistema de control para detener toda la maquinaria de forma inmediata
- Baliza de señalización – la baliza PATLITE LR4 instalada en el sistema permite señalar visualmente el estado de la línea de producción (operativa, en pause, fallo), y es necesaria para asegurar la seguridad en la planta
- Sensores de límite e interruptores – los interruptores de límite Omron instalados garantizan que los movimientos de la maquinaria se detienen al alcanzar posiciones críticas, evitando colisiones o daños
- Zona colaborativa de los ABB IRB – al ser robots colaborativos, los ABB IRB puede operar junto a personas bajo los límites establecidos por las normativas mencionadas en el apartado anterior (como la ISO/TS 15066)

5.0.3. Impacto en los puestos de trabajo

La automatización reduce el número de operarios en la línea de producción de 23 a 4 por ahorro de costos y porque queríamos dejar solo los trabajadores que realmente fueran necesarios. En línea con la ley 31/1995 y el compromiso ético de la empresa, Chocolat Supreme se compromete a gestionar esta transición de forma responsable mediante las siguientes acciones:

- Reubicación interna – queremos priorizar la reubicación de los operarios afectados en otras áreas como logística, control de calidad o atención a clientes.
- Plan de formación – obviamente, para la reubicación de operarios, estos recibirán formación específica en su área, sobre todo aquellos que pasen a supervisión del nuevo sistema
- Cumplimiento laboral – en caso de que no sea posible la reubicación, se actuará conforme al Estatuto de los Trabajadores y los convenios colectivos aplicables al sector industrial alimentario en España para asegurar que los operarios sean lo menos afectados posible

Capítulo 6

Conclusiones

La decisión de proponer este proyecto de automatización surge de una necesidad real y concreta que hemos identificado dentro de Chocolat Supreme: el proceso de empaquetado y paletizado, el cual representaba un cuello de botella que estaba afectando negativamente el rendimiento anual de la empresa. Tras analizar alternativas posibles y existentes, la automatización completa resultó ser la solución más sólida, tanto desde el punto de vista operativo como el económico, este siendo respaldado por un ROI del 402 % y un payback de 2,39 meses, todo por lo cual se justifica la viabilidad de nuestra propuesta.

Durante el desarrollo del proyecto nos encontramos con muchos desafíos que no habíamos anticipado. Empezando por el más grave, RoboDK resultó ser una herramienta más compleja y a la vez limitada de lo esperado, especialmente a la hora de programar acciones fluidas y simular la sincronización de los robots. También, otro reto significativo fue la integración entre la aplicación web y RoboDK a través de MQTT. El código que conectaba RoboDK con el broker MQTT que vimos en prácticas funcionaba únicamente con las credenciales específicas de la universidad, lo que hacía imposible utilizarlo directamente desde la interfaz web. Para resolver este problema tuvimos que diseñar puentes en Python para que hagan de intermediarios, adaptando toda la arquitectura de comunicación y realizando cambios considerables en la estructura original del sistema. Este proceso, aunque nos costó incontables horas, nos obligó a entender en profundidad cómo funciona la comunicación entre capas y nos dio una visión mucho más realista de los restos de integración en entornos industriales reales.

En cuanto a lo aprendido, este proyecto nos ha abierto los ojos (algo a la fuerza) en varios aspectos, especialmente en gestión de proyectos. Por un lado, hemos desarrollado un proyecto de robótica que van más allá de lo visto en teoría, enfrentándonos a la simulación real de movimientos, colisiones y coordinación entre robots. Por otro, la gestión del proyecto en sí – con fases, hitos, responsabilidades y uso de herramientas como Trello – nos ha dado una primera experiencia práctica de cómo se organiza un proyecto de ingeniería de este tamaño. Quizás lo más inesperado ha sido descubrir hasta qué punto Python y la ESP32-S3 pueden ser grandes herramientas versátiles – desde gestionar la lógica completa de un sistema industrial hasta actuar como nodo físico de comunicación en tiempo real, algo que al principio no imaginábamos que pudiera funcionar con tanta fluidez, permitiéndonos manejar los errores que nos encontrábamos en el camino.

El proyecto también ha puesto en práctica conocimientos de otras asignaturas de la carrera. Primero, los conocimientos de la asignatura de Sistemas Embebidos han sido fundamental para la programación de la ESP32-S3 y la implementación del protocolo MQTT, ya que nos ayudó a aplicar correctamente conceptos de comunicación entre dispositivos

físicos y software. Gestión de Datos e Integración ha estado también muy presente durante el proyecto, sobre todo en el diseño de la base de datos SQL, el estructurar las tablas de pedidos y stock, y la lógica de actualización constante de datos. Por último, Sistemas de Tiempo Real también ha sido muy relevante para la programación de tareas del backend, el seguimiento continuo del stock y la necesidad de que el sistema respondiera de forma inmediata a los eventos que podían suceder en la planta.

Como recomendaciones a futuras implementaciones del proyecto, creemos que lo mejor sería explorar una solución de conectividad MQTT más independiente para evitar nuestro problema con las credenciales de la universidad y evitar la necesidad de usar puentes de Python, así como ampliar más la interfaz web para ofrecer a los clientes una experiencia más completa y personal. En cuanto a la planta como tal, podríamos añadir la automatización del reabastecimiento físico de las cintas, cerrando así el ciclo productivo de forma totalmente automatizada.

Capítulo 7

Referencias

7.1. Descripción de la implementación

Para poder trabajar simultáneamente y tener un orden coherente, decidimos estructurar el sistema en tres ‘capas’ distintas. La primera, la capa física, está compuesta por la ESP32-S3, los robots y los elementos físicos de la planta – cintas, sensores y actuadores –, donde la ESP32-S3 actúa como puentes entre el software y el hardware, recibiendo instrucciones vía MQTT y ejecutándolas sobre los dispositivos. Después está la capa lógica, que se basa en el backend Python, el cual está dividido en módulos funcionales que cubren la gestión de stock, el procesamiento de pedidos, la generación de alertas y la publicación de mensajes en MQTT. Por último, la capa de datos consiste en la base de datos SQL que almacena constantemente los pedidos, el histórico de stock y los registros de producción, permitiendo tanto rastrear el producto durante el proceso como la generación de informes para el departamento financiero.

7.2. División en tareas y funciones

Aunque todas las integrantes del grupo contribuimos en todas las fases del proyecto, cada una hemos liderado un área técnica concreta. Empezando por Valentina Montoserín, como Ingeniera de Robótica y Simulación, ha sido la responsable del desarrollo de la simulación en RoboDK, incluyendo la programación de los movimientos de los ABB IRB y el robot de paletizado, definición de trayectorias y la validación del layout de la planta. Siguiendo con Ainhoa García, en su rol de Ingeniera de Automatización y Control, ha liderado la programación del backend Python, la lógica de control del sistema, la gestión de la comunicación MQTT con puentes de Python, la base de datos SQL, y el control de seguridad con sensores físicos. Rihab Baitar, por otro lado, como Programadora de Sistemas OT, ha sido la responsable de la creación de la página web con HTML, además de la creación del broker privado MQTT y la unión de la página con la base de datos y otros elementos del sistema. Finalmente, Claudia Castillo, como Ingeniera de Puesta en Marcha, ha coordinado la integración del microcontrolador ESP32-S3 además de su programación, la realización de la documentación y en general la integración de todos los componentes en el sistema.

7.3. Uso de la IA

Uso de la inteligencia artificial

La inteligencia artificial generativa se ha utilizado como herramienta de apoyo durante el desarrollo del proyecto, principalmente en la documentación, la resolución de dudas técnicas y la búsqueda de alternativas cuando la información disponible no era suficiente. No obstante, **no ha sustituido el trabajo del equipo**: el código funcional del sistema ha sido desarrollado, probado y validado por las integrantes del grupo.

En concreto, el diseño e implementación de la **base de datos PostgreSQL**, el **HTML y CSS** de la página web, la **programación de las secuencias en RoboDK** y el **firmware del ESP32** son trabajo propio del equipo. En cuanto a los scripts `mqtt_bridge.py`, `mqtt_station_relay.py` y `mqtt_industrial_bridge.py`, han podido ser implementados gracias a la inteligencia artificial para comprender cómo debían funcionar, ya que la arquitectura de intermediarios entre web, broker MQTT, base de datos y RoboDK se ha estudiado aún en clase y supuso uno de los aspectos técnicos más complejos del proyecto. No obstante, fueron revisados y comprendidos por las miembros del grupo. Mediante la IA hemos consultado, entre otras cuestiones, cómo estructurar la conexión a dos brokers distintos, cómo suscribirse y publicar en topics, cómo traducir mensajes JSON a operaciones SQL y cómo encadenar el flujo web $\rightarrow$ EMQX $\rightarrow$ PostgreSQL $\rightarrow$ relay $\rightarrow$ RoboDK.

En otros apartados, la IA se ha empleado de forma más puntual: corrección de errores, sugerencias de alternativas cuando algo no funcionaba, y aclaración de conceptos de MQTT o PostgreSQL que no habíamos visto con detalle. Para la memoria escrita en LATEX, también hemos recibido ayuda con comandos de maquetación que desconocíamos; la redacción del contenido ha sido elaborada por las distintas miembros del grupo.

Los resultados financieros, las pruebas de funcionamiento y la validación del sistema han sido calculados y comprobados por las autoras. Completar el proyecto a tiempo ha requerido un proceso iterativo de prueba y error, especialmente en la comunicación entre capas.

En conclusión, el uso de inteligencia artificial o ha consistido en que hiciera el trabajo por nosotras, sino en orientarnos en materias y procesos que aún no dominábamos, sobre todo los puentes Python. Gracias a ello hemos podido adentrarnos en protocolos de mensajería, persistencia en base de datos e integración web-industrial, interesarnos por ellos y, lo más importante, entender cómo funciona el sistema, supervisar y defenderlo con criterio propio.

7.4. Referencias bibliográficas

1,27cm1

ABB. (s. f.). *IRB 4400*. ABB Robotics. <https://one.robotics.abb.com/es/robots/p/IRB-4400>

ABB. (s. f.). *CRB 15000*. ABB. <https://new.abb.com/products/robotics/es/robots/robots-colaborativos/gofa-crb-15000>

Alibaba.com. (s. f.). *Máquina automática para envolver cajas con cinta*. Alibaba. <https://spanish.alibaba.com/product-detail/M%C3%A1quina-Autom%C3%A1tica-para-Envolver-Cajas.html>

Eclipse Foundation. (s. f.). *Eclipse Mosquitto documentation*. Eclipse Mosquitto. <https://mosquitto.org/documentation/>

España. (1995, 8 de noviembre). *Ley 31/1995, de 8 de noviembre, de prevención de Riesgos Laborales*. Boletín Oficial del Estado. <https://www.boe.es/buscar/act.php?id=BOE-A-1995-24292>

España. (1997, 18 de julio). *Real Decreto 1215/1997, por el que se establecen las disposiciones mínimas de seguridad y salud para la utilización de los equipos de trabajo*. Boletín Oficial del Estado. <https://www.boe.es/buscar/act.php?id=BOE-A-1997-17824>

EVS International. (2026). *How much does an industrial robot cost? Pricing guide 2026*. EVSINT. <https://www.avsint.com/es/how-much-does-an-industrial-robot-cost-pricing>

Flintec. (s.f.). *Accurex TX-Tiger*. Flintec. <https://flintec.es/accurex-tx-tiger/>

Iberoptics. (s.f.). *Hikvision ID6000 code reader*. Iberoptics. https://www.iberoptics.com/es/lectores-de-codigos/hik_lector-de-codigos-id6000-4551.html

International Organization for Standardization. (2011). *ISO 10218-1:2011 Robots and robotic devices — Safety requirements for industrial robots*. ISO. <https://www.iso.org/standard/51330.html>

International Organization for Standardization. (2016). *ISO/TS 15066:2016 Robots and robotic devices — Collaborative robots*. ISO. <https://www.iso.org/standard/62996.html>

Mozilla Developer Network. (s.f.). *HTML: HyperText Markup Language*. MDN Web Docs. <https://developer.mozilla.org/es/docs/Web/HTML>

Omron. (s.f.). *D4N-1120 limit switch*. Farnell España. <https://es.farnell.com/omron/d4n-1120/interruptor-l-mite-palanca-rotativa/dp/1125852>

OnRobot. (s.f.). *VGP20 vacuum gripper*. OnRobot. <https://onrobot.com/es/productos/vgp20>

OnRobot. (s.f.). *VG10 electric vacuum gripper*. OnRobot. <https://onrobot.com/es/productos/pinza-electrica-de-vacio-vg10>

PATLITE Corporation. (s.f.). *LR4-302WJNW-RYG signal tower*. Automation24. <https://www.automation24.es/baliza-de-senalizacion-patlite-lr4-302wjnw-ryg>

Parlamento Europeo y Consejo de la Unión Europea. (2006, 17 de mayo). *Directiva 2006/42/CE del Parlamento Europeo y del Consejo, relativa a las máquinas*. Diario Oficial de la Unión Europea. <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32006L0042>

Parlamento Europeo y Consejo de la Unión Europea. (2004, 30 de abril). *Reglamento (CE) n.º 853/2004 relativo a la higiene de los productos alimenticios*. Diario Oficial de la Unión Europea. <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32004R0852>

Random Nerd Tutorials. (s.f.). *ESP32 MQTT — Publish and Subscribe with Arduino IDE*. Random Nerd Tutorials. <https://randomnerdtutorials.com/esp32-mqtt-publish-subscribe>

Real Python. (s.f.). *Python sockets programming*. Real Python. <https://realpython.com/python-sockets/>

Research Cash Lab. (s.f.). *Cómo se hace un análisis financiero de un proyecto*. Research Cash Lab. <https://www.researchcashlab.com/como-se-hace-un-analisis-financiero-pro>

Saraiva, P. (2019, 28 de febrero). *MQTT in Python with Paho*. Towards Data Science. <https://towardsdatascience.com/mqtt-in-python-with-paho-ab98e71a63c4>

VEVOR. (s.f.). *Transportador de correa motorizado 1498,6 x 198,1 mm*. VEVOR España. https://www.vevor.es/cinta-transportadora-c_10439/vevor-transportador-de-correa-010617620264

VEVOR. (s.f.). *Transportador de correa motorizado 1498,6 x 398,8 mm*. VEVOR España. https://www.vevor.es/cinta-transportadora-c_10439/vevor-transportador-de-correa-

010184700421

W3Schools. (s. f.). *SQL tutorial*. W3Schools. <https://www.w3schools.com/sql/>

Capítulo 8

Anexos

8.1. Anexo I. Manual de instalación y funcionamiento

8.1.1. 1. Requisitos previos

Antes de ejecutar la aplicación es necesario disponer de los siguientes elementos:

Python 3.11 o superior instalado en el sistema.

RoboDK instalado.

Acceso al broker MQTT utilizado por el proyecto.

Copia completa de la carpeta del proyecto (Carpeta de Instalacion.zip)

LINK:https://github.com/Ainhua0608/Trabajo_Final_PR2/blob/main/Carpeta%20de%20Instalacion.zip

Verificación de Python

Para comprobar que Python está correctamente instalado, abrir una terminal (CMD o PowerShell) y ejecutar:

```
python --version
```

Si la instalación es correcta, se mostrará una versión de Python (por ejemplo, Python 3.11 o superior).

En caso de que el comando no sea reconocido, deberá instalarse Python desde la página oficial:

<https://www.python.org/downloads/>

Durante la instalación es imprescindible activar la opción “Add Python to PATH”.

8.1.2. 2. Configuración del proyecto

Una vez descargada la carpeta del proyecto, debe verificarse que el archivo .env contiene las credenciales correctas para la conexión con la base de datos PostgreSQL y los servidores utilizados por la aplicación.

Estas credenciales deben coincidir con las proporcionadas para el entorno de trabajo del proyecto.

8.1.3. 3. Instalación de dependencias

La primera vez que se ejecute la aplicación será necesario instalar las librerías requeridas.

Abrir una terminal dentro de la carpeta principal del proyecto y ejecutar:

```
python -m pip install -r requirements.txt
```

Este comando instalará automáticamente todas las dependencias necesarias, entre ellas:

paho-mqtt

python-dotenv

certifi

pg8000 (utilizada por el puente MQTT cuando sea necesario)

La instalación únicamente deberá realizarse una vez por equipo.

8.1.4. 4. Configuración de RoboDK

Abrir el archivo .rdk correspondiente al proyecto.

Las secuencias desarrolladas se encuentran organizadas dentro de RoboDK y pueden ejecutarse desde los programas internos de la aplicación.

En caso de utilizar la integración MQTT directamente desde RoboDK, deberá verificarse que el programa correspondiente esté suscrito al tópico de comunicaciones utilizado por el proyecto:

```
giirob/pr2/station/demo/commands
```

Esta etapa es opcional, ya que la comunicación principal se realiza mediante el puente MQTT descrito en la siguiente sección.

8.1.5. 5. Ejecución del puente MQTT

Para establecer la comunicación entre RoboDK y el resto de componentes del sistema, debe ejecutarse el programa de enlace MQTT.

Desde una terminal ubicada en la carpeta del proyecto ejecutar:

```
python mqtt_station_relay.py
```

Una vez iniciado, el programa permanecerá en ejecución escuchando mensajes MQTT.

Importante: la ventana de la terminal debe permanecer abierta durante toda la prueba o simulación.

8.1.6. 6. Funcionamiento del sistema

Con el puente MQTT en ejecución, el sistema está preparado para recibir pedidos.

El funcionamiento es el siguiente:

Un usuario realiza un pedido desde la aplicación web o desde el sistema de control.

El pedido genera un mensaje MQTT con el identificador de la combinación seleccionada.

El puente MQTT recibe dicho mensaje.

RoboDK interpreta la combinación recibida y ejecuta automáticamente la secuencia correspondiente.

Se inicia la simulación de producción, incluyendo los movimientos de transporte, Pick and Place y paletizado.

Para supervisar los mensajes intercambiados puede utilizarse una herramienta como MQTTX, suscribiéndose al tópico:

```
giirob/pr2/station/demo/commands
```

8.1.7. 7. Verificación y resolución de problemas

Si el sistema no funciona correctamente, comprobar los siguientes puntos:

Python no responde

Verificar que Python está instalado ejecutando:

```
python --version
```

Dependencias no instaladas

Comprobar que se ha ejecutado correctamente:

```
python -m pip install -r requirements.txt
```

Error de configuración

Verificar que el archivo .env contiene las credenciales correctas para la base de datos y los servicios utilizados.

Error en el puente MQTT

Comprobar que la terminal donde se ejecuta:

```
python mqtt_station_relay.py
```

permanece abierta y no muestra mensajes de error.

RoboDK no recibe comandos

Verificar que RoboDK se encuentra en ejecución y conectado al mismo broker MQTT utilizado por el proyecto:

```
mqtt.dsic.upv.es
```

Asimismo, comprobar que el tópico configurado coincide con el utilizado por el sistema.

8.2. Anexo II. Relacion con PRA y GDI

En el desarrollo del sistema se han utilizado estructuras de datos vistas en la asignatura PRA, junto con un modelo relacional en PostgreSQL para la persistencia de la información.

En la capa lógica (Python y JavaScript), las listas se emplean para gestionar colecciones ordenadas de elementos como el carrito de la compra, las líneas de pedido o las tarjetas de pago asociadas a los clientes. Asimismo, los diccionarios se utilizan como estructura principal para el mapeo de información entre capas, permitiendo acceso directo y eficiente a datos clave del sistema.

Un aspecto relevante del diseño es la gestión de las combinaciones de sabores del sistema PR2. Estas combinaciones se representan mediante estructuras tipo árbol conceptual, que posteriormente se materializan en memoria como diccionarios de acceso directo. En particular, las estructuras `COMBINACION_A_NUMERO` y `RUTA_A_NUMERO` permiten traducir cada combinación de sabores en un identificador numérico (`numeroCombi`) en tiempo constante, el cual se utiliza para activar la secuencia correspondiente en RoboDK.

En paralelo, el sistema de base de datos PostgreSQL constituye la base del almacenamiento persistente. El modelo relacional se organiza en torno a entidades principales como `clientes`, `pedidos`, `lineas_pedido`, `stock_sabores`, `envios` y `pedidos_combinacion`. Esta estructura permite mantener la integridad de los datos, asegurar la trazabilidad de los pedidos y realizar operaciones atómicas durante la confirmación de un pedido.

En conjunto, la arquitectura combina estructuras de datos en memoria (listas, diccionarios y mapeos de combinaciones) para el procesamiento en tiempo real, con un sistema relacional en PostgreSQL para la persistencia y consulta de la información del sistema.

8.3. Anexo III. Puesta en marcha del sistema de sensores (ESP32 + RoboDK)

Este anexo describe cómo arrancar la **planta de seguridad** del proyecto: ESP32-S3 con botones y sensor ultrasónico, puente Python hacia el broker UPV y simulación en RoboDK. No cubre el flujo completo de pedidos web (véase Anexo I); aquí el foco son los comandos `PER`, `STOP`, `SLOW` y `NORMAL`.

8.3.1. Componentes necesarios

- ESP32-S3 programado con `sabor_industrial_mqtt.ino`.
- Dos pulsadores (GPIO 4 = `PER`, GPIO 5 = `STOP`) y sensor ultrasónico HC-SR04 (TRIG = GPIO 6, ECHO = GPIO 7).
- PC con Python 3.11+, conexión WiFi y acceso a EMQX Cloud y al broker UPV (`mqtt.dsic.upv.es`).
- RoboDK con el proyecto `.rdk` y el script `MQTT_listener.py`.
- Script `mqtt_industrial_bridge.py` en ejecución en el PC.

8.3.2. Flujo de comunicación

ESP32 → EMQX Cloud → `mqtt_industrial_bridge.py`
→ `mqtt.dsic.upv.es` → `MQTT_listener.py` (RoboDK)

8.3.3. Paso 1: Configurar el ESP32 (secrets.h)

En la carpeta `choco_esp32_mqtt/sabor_industrial_mqtt/`, copiar la plantilla:

```
copy secrets.h.example secrets.h
```

Editar `secrets.h` con la red WiFi y las credenciales MQTT:

```
#define WIFI_SSID "UPV-PSK"
#define WIFI_PASSWORD "tu_clave_wifi"

#define MQTT_HOST "kf12786f.ala.us-east-1.emqxsl.com"
#define MQTT_PORT 8883
#define MQTT_USER "web_choco"
#define MQTT_PASS "... " // misma contraseña que en .env (EMQX)
#define MQTT_USE_TLS 1

#define MQTT_TOPIC_COMMANDS "giirob/pr2/station/demo/commands"
```

Importante:

- El ESP32 debe estar en la **misma red WiFi** que el PC (o en una red con acceso a Internet).
- Usar WiFi **2,4 GHz** (el ESP32 no suele conectar bien a 5 GHz).
- **No subir secrets.h** a GitHub; solo `secrets.h.example`.

8.3.4. Paso 2: Librerías Arduino necesarias

En Arduino IDE, instalar desde *Gestor de librerías*:

- **PubSubClient** (Nick O’Leary) — cliente MQTT.
- **WiFi** — incluida con el core del ESP32 (instalar soporte *esp32* por Espressif si falta).

Placa recomendada: **ESP32S3 Dev Module**. Monitor serie: **115200 baudios**.

8.3.5. Paso 3: Subir el sketch al ESP32

1. Abrir `sabor_industrial_mqtt.ino` en Arduino IDE.
2. Seleccionar placa ESP32-S3 y el puerto COM correcto.
3. Compilar y subir (*Subir*).
4. Abrir Monitor serie (115200) y pulsar reset si hace falta.

Salida esperada en el Monitor serie:

```
=== ESP32 botones PER / STOP ===
Conectando WiFi UPV-PSK...
WiFi OK IP: 192.168.x.x
Conectando MQTT kf12786f.ala.us-east-1.emqxsl.com:8883...
MQTT OK
Listo: GPIO 4=PER, GPIO 5=STOP
Topic commands: giirob/pr2/station/demo/commands
```

Al pulsar el botón PER (GPIO 4):

```
Boton PER (GPIO 4)
Publish [giirob/pr2/station/demo/commands] -> PER (OK)
```

Al acercar un objeto al ultrasonido (< 7 cm):

```
Ultrasonido: 5.23 cm -> SLOW
Publish [...] -> SLOW (OK)
```

8.3.6. Paso 4: Configurar el PC (.env)

En la carpeta raíz del proyecto:

```
copy .env.example .env
python -m pip install -r requirements.txt
```

Rellenar al menos en .env:

```
MQTT_URL=wss://kf12786f.ala.us-east-1.emqxsl.com:8084/mqtt
MQTT_USERNAME=web_choco
MQTT_PASSWORD=...

PROYECTOS_MQTT_HOST=mqtt.dsic.upv.es
PROYECTOS_MQTT_PORT=1883
PROYECTOS_MQTT_USERNAME=giirob
PROYECTOS_MQTT_PASSWORD=... # contrasea broker UPV (obligatorio)

STATION_COMMANDS_TOPIC=giirob/pr2/station/demo/commands
EMQX_SOURCE_TOPICS=giirob/pr2/station/demo/commands
```

8.3.7. Paso 5: Arrancar el puente industrial

En una terminal, desde la carpeta del proyecto:

```
python mqtt_industrial_bridge.py
```

Salida esperada:

```
Puente ESP32 -> UPV activo. Ctrl+C para salir.
EMQX OK -> subscribed: giirob/pr2/station/demo/commands
UPV OK -> reenvio a: giirob/pr2/station/demo/commands (PER / STOP / SLOW /
NORMAL)
```

Al pulsar PER en el ESP32, la terminal debe mostrar:


```
EMQX [giirob/pr2/station/demo/commands] 'PER'
-> UPV [giirob/pr2/station/demo/commands] 'PER'
```

Si aparece Falta PROYECTOS_MQTT_PASSWORD en .env, rellenar esa variable y volver a ejecutar.

8.3.8. Paso 6: Iniciar RoboDK

1. Abrir el proyecto .rdk de la estación.
2. Comprobar que existen los programas Script_P&P_paletizado, P&P_per, P&P_c1 y P&P_c2 en el árbol (o equivalentes).
3. Ejecutar **solo** MQTT_listener.py (tecla F5). **No** ejecutar a la vez los scripts de paletizado a mano.

Salida esperada en la consola de RoboDK:

```
Escuchando giirob/pr2/station/demo/commands
OK industrial -> Script_P&P_paletizado
OK personal -> P&P_per
MQTT conectado. Arrancando Script_P&P_paletizado
```

8.3.9. Paso 7: Orden de encendido recomendado

1. Encender ESP32 (WiFi + MQTT OK en monitor serie).
2. Ejecutar python mqtt_industrial_bridge.py en el PC.
3. Abrir RoboDK y F5 en MQTT_listener.py.
4. Probar botones y ultrasonido.

8.3.10. Comportamiento esperado al probar sensores

Cuadro 8.1: Comandos ESP32 y respuesta en RoboDK

Acción	Comando MQTT	Efecto en RoboDK
Pulsar botón PER	PER	Para industrial y ejecuta P&P_per
Pulsar botón STOP	STOP	Parada de emergencia (RDK.Stop())
Mano < 7 cm al ultrasonido	SLOW	Velocidad C1/C2 reducida (20°/s)
Alejar mano > 9 cm	NORMAL	Velocidad C1/C2 normal (60°/s)

En la consola de RoboDK debería aparecer, por ejemplo:

```
MQTT comando: PER
*** PER: parar industrial -> personalizado ***
Ejecutando: P&P_per
```

8.3.11. Solución de problemas

ESP32 no conecta a WiFi

Comprobar SSID/contraseña en `secrets.h`, red 2,4 GHz y que el monitor serie esté a 115200 baudios.

ESP32: MQTT fallo, `rc=...`

Revisar `MQTT_HOST`, `MQTT_PORT` (8883 con TLS), usuario y contraseña EMQX. Comprobar acceso a Internet.

Puente Python no arranca

Faltan variables en `.env`. Ejecutar `python diagnostico.py` para ver el detalle.

Puente activo pero RoboDK no reacciona

- Verificar que `mqtt_industrial_bridge.py` muestra `->UPV [...] 'PER'` al pulsar.
- Comprobar que RoboDK está conectado a `mqtt.dsic.upv.es` (credenciales en `MQTT_listener.py`)
- Solo debe estar en ejecución `MQTT_listener.py`, no otros listeners duplicados.

PER no para al instante

RoboDK termina el movimiento en curso antes de parar; es comportamiento normal. Para parada más rápida habría que modificar los scripts de paletizado.

Comando ignorado en el puente

Solo se reenvían PER, STOP, SLOW y NORMAL. Cualquier otro payload muestra Ignorado (esperado PER/STOP/SLOW/NORMAL).

8.3.12. Resumen

Para que funcione el sistema de sensores hacen falta **tres piezas activas**: ESP32 publicando en EMQX, `mqtt_industrial_bridge.py` reenviando al broker UPV, y `MQTT_listener.py` en RoboDK interpretando los comandos. Si las tres muestran los mensajes esperados de las secciones anteriores, la cadena ESP32 → RoboDK está operativa.

8.4. Anexo IV

Inversión inicial		
Lista de componentes	Valor monetario	Tipo
ABB IRB 4400/L30 M2000	14.500,00 €	Activo no corriente
ABB CRB 1500	32.000,00 €	Activo no corriente
VG20 VACUUM GRIPPER ONROBOT	6.530,00 €	Activo no corriente
VG10 VACUUM GRIPPER ONROBOT	4.201,00 €	Activo no corriente
VEVOR Transportador de Correa Motorizado 1498,6x198,1 mm	256,90 €	Activo no corriente
VEVOR Transportador de Correa Motorizado 1498,6x398,8 mm	256,90 €	Activo no corriente
Jining Makeway Máquina Automática para Envolver Cajas con Cinta de Espuma para Sellado y Empaquetado	781,38 €	Activo no corriente
Baliza de señalización PATLITE LR4-302WJNW-RYG	122,57 €	Activo no corriente
LECTOR DE CÓDIGOS HIK - ID6000	350,90 €	Activo no corriente
Báscula de suelo industrial Accurex TX-TIGER	350,90 €	Activo no corriente
Interruptor de Límite, IP67, Palanca con Roldana, SPST-NO, SPST-NC, 10A, 240V, 5N, D4N Series	40,52 €	Activo no corriente
Total materiales	59.391,07 €	€
Otros gastos (instalación, configuración, capacitación, seguridad)	32.500,00 €	

Figura 8.1: analisis financiero

Gastos antes de la automatización	Datos	Importe anual (€)		Gastos despues de la automatización	Datos	Importe anual (€)
Número de los operarios	23			Número de los operarios	4	
Salario bruto anual por operario	24.317,00 €			Salario bruto anual por operario	24.317,00 €	
Total salarios brutos		559.291,00 €		Total salarios brutos		97.268,00 €
Costes Sociales (30% de seguridad social)	30,00%	167.787,30 €		Costes Sociales (30% de seguridad social)	30,00%	29.180,40 €
Subtotal personal		727.078,30 €		Subtotal personal		126.448,40 €
Mermas y errores				Mermas y errores		
mermas por error anual	3,00%			mermas por error anual	1,00%	
Producción manual anual (cajas grandes)	26.400,00 €			Producción automática anual (cajas grandes)	66.000,00	
Producción manual anual (cajas grandes)	15,00 €			Producción automática anual (cajas grandes)	37,50 €	
Pérdidas por mermas		11.880,00 €		Pérdidas por mermas		24.750,00 €
TOTAL GASTOS ANUALES MANUAL		738.958,30 €		TOTAL GASTOS ANUALES TRAS AUTOMATIZACIÓN		277.648,80 €

Figura 8.2: analisis financiero

RETORNO DE INVERSIÓN (ROI) Y PAYBACK	
Inversión total inicial	91.891,07 €
Ahorro neto anual	461.311,50 €
ROI (Retorno sobre la inversión)	402%
Payback (meses)	2,39

Figura 8.3: analisis financiero